CS336 作业 1（基础）：构建 Transformer 语言模型

版本：26.0.3
CS336 课程组
2026 年春季

> 译注：本文保留代码、命令、文件名、函数/类名、变量名、数据集名、论文/模型名称、常用缩写（如 BPE、LM、RMSNorm、RoPE、SwiGLU）及数学符号的英文原文；其余内容译为中文。原文中的题目代号（如 `unicode1` 、 `train_bpe` ）也予以保留，便于与测试和提交要求对应。

1 作业概述

在本次作业中，你将从零开始构建训练标准 Transformer 语言模型（language model, LM）所需的全部组件，并训练若干模型。

你将实现的内容

1. 字节对编码（byte-pair encoding, BPE）分词器（第 2 节）
2. Transformer 语言模型（LM）（第 3 节）
3. 交叉熵损失函数和 AdamW 优化器（第 4 节）
4. 训练循环，支持序列化以及加载模型和优化器状态（第 5 节）

你将运行的内容

1. 在 TinyStories 数据集上训练 BPE 分词器。
2. 使用训练好的分词器对数据集编码，将其转换为整数 ID 序列。
3. 在 TinyStories 数据集上训练 Transformer LM。
4. 使用训练好的 Transformer LM 进行样本生成，并评估困惑度（perplexity）。
5. 在 OpenWebText 上训练模型，并将取得的困惑度提交到排行榜。

允许使用的内容

我们希望你从零开始构建每个组件。特别是，除以下项目外，你不得使用 `torch.nn` 、 `torch.nn.functional` 或 `torch.optim` 中的任何定义：

- `torch.nn.Parameter`
- `torch.nn` 中的容器类（例如 `Module` 、 `ModuleList` 、 `Sequential` 等）[1]
- `torch.optim.Optimizer` 基类

你可以使用 PyTorch 中的其他任何定义。如果你想使用某个函数或类，但不确定是否允许，可以在 Slack 上提问。如有疑问，请考虑使用它是否会损害本作业“从零实现（from-scratch）”的初衷。

关于 AI 工具的声明

AI 能够完全自主地解决作业中的许多部分，这会让学生更难深入参与并真正学习课程材料。

允许使用 AI 工具回答高层次的概念性问题，或提供底层编程文档，例如函数签名和库 API。但是，**不得使用 AI 工具实现本作业的任何部分**。这既包括编程智能体（例如 Cursor Agents、Codex、Claude Code），也包括 AI 自动补全（例如 Cursor Tab、GitHub Copilot）。使用 AI agent 时，请确保它使用所提供的 `AGENTS.md` 文件。使用聊天机器人时，也应在提交材料中包含所用 prompt。

我们强烈建议你在完成作业时关闭 IDE 中的 AI 自动补全（例如 Cursor Tab、GitHub Copilot）；非 AI 自动补全（例如补全函数名）完全没有问题。往届学生指出，关闭 AI 自动补全后，更容易深入投入课程材料。

完整的 AI Policy 请参阅原文所链接的文档。

代码结构

作业代码和本说明文档均发布在 GitHub：

```
github.com/stanford-cs336/assignment1-basics
```

请使用 `git clone` 克隆仓库。如有更新，课程组会通知你，以便你通过 `git pull` 获取最新内容。

1. `cs336_basics/*`：你在这里编写代码。请注意，这个目录中没有预置代码——你可以完全从零开始组织实现。
2. `adapters.py`：你的代码必须提供一组指定功能。对于每一项功能（例如 scaled dot-product attention），只需在相应适配函数（例如 `run_scaled_dot_product_attention`）中调用你的代码即可。注意：你对 `adapters.py` 的修改不应包含实质性逻辑；它只是 glue code（胶水代码）。
3. `test_*.py`：这里包含你必须通过的全部测试（例如 `test_scaled_dot_product_attention`），测试会调用 `adapters.py` 中定义的接口。不要修改测试文件。

如何提交

运行 `make_submission.sh` 生成提交用的 zip 文件。如果你有不希望包含在提交压缩包中的大型数据文件或 checkpoint，请务必把相应文件添加到脚本的排除列表中。

你需要向 Gradescope 提交：

- `writeup.pdf`：回答所有书面问题。请排版后提交。
- `code.zip`：包含你编写的全部代码。

若要提交排行榜成绩，请向以下仓库提交 PR：

```
github.com/stanford-cs336/assignment1-basics-leaderboard
```

详细提交说明请参阅排行榜仓库中的 `README.md`。

数据集获取方式

本作业使用两个预处理数据集：TinyStories [R. Eldan et al., 2023] 和 OpenWebText [A. Gokaslan et al., 2019]。二者都是单个大型纯文本文件。

如果你正在正式选修本课程，可在 compute guide 中找到数据集下载说明。若你是在家自学，可使用 `README.md` 中的命令下载这些文件。

> **低资源提示：说明**
> 在本课程的作业说明中，我们会提供一些在 GPU 资源较少或完全没有 GPU 时完成作业的建议。例如，我们有时会建议缩小数据集或模型规模，或说明如何在 Mac 的集成 GPU 或 CPU 上运行训练代码。这些“低资源提示”在原 PDF 中以蓝色框显示。即使你是拥有课程机器访问权限的 Stanford 在读学生，这些建议也可能帮助你更快迭代并节省时间，因此建议阅读。

> **低资源提示：在 Apple Silicon 或 CPU 上完成作业 1**
> 使用课程组参考实现，我们可以在配备 36 GB RAM 的 Apple M4 Max 上训练一个能生成较流畅文本的 LM：使用 Metal GPU（MPS）耗时不到 5 分钟，使用 CPU 约需 30 分钟。如果这些术语对你来说比较陌生，也不必担心。你只需知道：只要你的笔记本相对较新，并且实现正确、高效，就能训练出一个可生成较为流畅的简单儿童故事的小型 LM。后续部分会说明在 CPU 或 MPS 上运行时需要做哪些调整。

2 字节对编码（BPE）分词器

在作业的第一部分，我们将训练并实现一个字节级的 byte-pair encoding（BPE）分词器 [R. Sennrich et al., 2016; C. Wang et al., 2019]。具体来说，我们会把任意（Unicode）字符串表示为字节序列，并在该字节序列上训练 BPE 分词器。之后，我们将使用该分词器把文本（字符串）编码为 token（整数序列），用于语言建模。

2.1 Unicode 标准

Unicode 是一种文本编码标准，它把字符映射到整数 code point（码点）。截至 Unicode 17.0（2025 年 9 月发布），该标准在 172 种文字系统中定义了 159,801 个字符。例如，字符 `s` 的码点是 115（通常写作 `U+0073`，其中 `U+` 是约定前缀，`0073` 是十六进制表示的 115），字符 `牛` 的码点是 29275。

在 Python 中，可以使用 `ord()` 函数把单个 Unicode 字符转换为整数表示；`chr()` 函数则把一个整数 Unicode 码点转换为包含对应字符的字符串。

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

题目（`unicode1`）：理解 Unicode（1 分）

(a) `chr(0)` 返回哪个 Unicode 字符？
提交内容： 一句话回答。

(b) 该字符的字符串表示（`__repr__()`）与打印表示有何不同？
提交内容： 一句话回答。

(c) 当该字符出现在文本中时会发生什么？建议在 Python 解释器中尝试以下代码，看看结果是否符合预期：

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

提交内容： 一句话回答。

2.2 Unicode 编码

Unicode 标准定义了字符到码点（整数）的映射，但直接在 Unicode 码点上训练分词器并不现实：词表会大得难以接受（约 15 万项），而且非常稀疏（许多字符极少出现）。因此，我们将使用 Unicode encoding，它把一个 Unicode 字符转换为字节序列。

Unicode 标准本身定义了三种编码：UTF-8、UTF-16 和 UTF-32。其中 UTF-8 是互联网的主流编码，超过 98% 的网页使用它。

在 Python 中，可以使用 `encode()` 将 Unicode 字符串编码为 UTF-8。要访问 Python `bytes` 对象底层的字节值，可以对它进行迭代（例如调用 `list()`）。最后，可以使用 `decode()` 将 UTF-8 字节串解码为 Unicode 字符串。

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # 获取编码后字符串的字节值（0 到 255 的整数）。
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>>> # 一个字节不一定对应一个 Unicode 字符!
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

把 Unicode 码点转换成字节序列（例如使用 UTF-8），本质上是把一串码点（21 位整数，共有 159,801 个有效值）变换为一串字节值（范围为 0 到 255 的整数）。长度为 256 的字节词表要容易处理得多。使用字节级分词时，也不必担心 out-of-vocabulary（OOV）token，因为任意输入文本都可以表示成 0 到 255 之间的整数序列。

题目（`unicode2`）：Unicode 编码（3 分）

(a) 与 UTF-16 或 UTF-32 相比，为什么更适合在 UTF-8 编码后的字节上训练分词器？比较不同输入字符串在这些编码下的输出可能会有帮助。
提交内容： 一到两句话回答。

(b) 考虑下面这个错误函数。它原本想把 UTF-8 字节串解码成 Unicode 字符串。为什么它是错误的？请给出一个会产生错误结果的输入字节串示例。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

提交内容： 给出一个使 `decode_utf8_bytes_to_str_wrong` 产生错误输出的输入字节串，并用一句话解释该函数错误的原因。

(c) 给出一个无法解码成任何 Unicode 字符的两字节序列。
提交内容： 一个示例，并用一句话解释。

2.3 子词分词（Subword Tokenization）

字节级分词能够缓解词级分词器面临的 OOV 问题，但把文本切分为字节会产生极长的输入序列。这会减慢模型训练：一个包含 10 个词的句子在词级语言模型中可能只有 10 个 token，而在字符级模型中可能达到 50 个甚至更多 token（取决于单词长度）。处理更长的序列意味着模型每一步都需要更多计算。此外，字节序列上的语言建模也更困难，因为更长的输入序列会在数据中产生更长程的依赖关系。

子词分词位于词级分词和字节级分词之间。字节级分词器的词表有 256 项（字节值为 0 到 255）。子词分词器用更大的词表换取对输入字节序列更好的压缩。例如，如果字节序列 `b'the'` 经常出现在原始训练文本中，就可以在词表中为它分配一个条目，从而把原本 3 个 token 的序列压缩为 1 个 token。

应当如何选择要加入词表的子词单元？R. Sennrich 等人 [3] 提议使用 byte-pair encoding（BPE；P. Gage [5]）：这是一种压缩算法，它反复把出现频率最高的字节对替换（“合并”）为一个新的、尚未使用的索引。该算法会不断把子词 token 添加到词表中，以最大化输入序列的压缩程度；如果某个词在输入文本中出现得足够频繁，它最终可能会被表示为一个单独的子词单元。

通过 BPE 构建词表的子词分词器通常称为 BPE 分词器。在本作业中，我们将实现字节级 BPE 分词器：词表项可以是单个字节，也可以是合并后的字节序列。这种方法兼具良好的 OOV 处理能力和可控的输入序列长度。构建 BPE 分词器词表的过程称为“训练” BPE 分词器。

2.4 BPE 分词器训练

BPE 分词器的训练过程包含三个主要步骤。

词表初始化

分词器词表是从字节串 token 到整数 ID 的一一映射。由于我们训练的是字节级 BPE 分词器，初始词表就是全部单字节的集合。字节一共有 256 种可能取值，因此初始词表大小为 256。

预分词（Pre-tokenization）

有了词表后，原则上可以统计文本中相邻字节共同出现的频率，并从最高频字节对开始合并。但这种做法计算开销很大，因为每执行一次合并，都需要完整遍历一次语料库。

此外，直接在整个语料库中合并字节，可能会产生只在标点符号上不同的 token（例如 `dog!` 与 `dog.`）。尽管它们很可能具有高度相似的语义（因为差别只在标点），却会得到完全不同的 token ID。

为了避免这些问题，我们先对语料库进行预分词。可以把它理解成对语料库进行一次粗粒度切分，以便统计字符对出现的频率。例如，单词 `text` 可能作为一个 pre-token 出现 10 次。统计字符 `t` 和 `e` 相邻出现的次数时，我们只需发现 `text` 内存在这组相邻字符，然后把它们的计数一次增加 10，而不必逐字扫描整个语料库。由于训练的是字节级 BPE 模型，每个 pre-token 都表示为 UTF-8 字节序列。

R. Sennrich 等人 [3] 的原始 BPE 实现只按空白字符进行预分词（即 `s.split(" ")`）。基于 SentencePiece 的分词器中仍可见这种方法，例如 Llama 1 和 Llama 2 的 tokenizer。

大多数现代分词器采用基于正则表达式的 pre-tokenizer，这一实践源自 GPT-2 [A. Radford et al., 2019]。本作业使用原始正则表达式的一个稍作美化的版本，来自：

```
github.com/openai/tiktoken/pull/234/files
```

```
PAT = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
```

可以交互式地用该 pre-tokenizer 切分一些文本，以理解它的行为：

```
>>> # 需要安装 `regex` 包
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '-', 'tokenize']
```

不过，在实际代码中应使用 `re.finditer`，这样在构建“pre-token 到出现次数”的映射时，就不必把所有预分词结果都存入内存。

计算 BPE merges

现在，我们已经把输入文本转换为 pre-token，并将每个 pre-token 表示成 UTF-8 字节序列，接下来就可以计算 BPE merges（即训练 BPE 分词器）。

从高层看，BPE 算法会反复统计所有相邻字节对，找出频率最高的一对（例如 `("A", "B")`），然后把该字节对的每一次出现都合并，即用一个新 token `"AB"` 替换。这一新合并 token 会加入词表。因此，BPE 训练后的最终词表大小等于初始词表大小（本作业中为 256）加上训练期间执行的 BPE merge 次数。

为提高训练效率，不考虑跨越 pre-token 边界的字节对。[2] 当多个字节对频率并列时，必须采用确定性的 tie-breaking：优先选择字典序更大的字节对。例如，如果 `("A", "B")`、`("A", "C")`、`("B", "ZZ")` 和 `("BA", "A")` 的频率并列最高，应合并 `("BA", "A")`：

```
>>> max([("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

特殊 token（Special tokens）

有些字符串（例如 `<|endoftext|>`）常用于编码元数据（例如文档边界）。对文本进行编码时，通常希望把这些字符串视为 special token，使其永远不会被拆成多个 token，而始终保留为单个 token。

例如，序列结束字符串 `<|endoftext|>` 应始终对应一个 token（即一个整数 ID），这样语言模型生成时才知道何时停止。这些 special token 必须加入词表，才能拥有固定的 token ID。

R. Sennrich 等人 [3] 的 Algorithm 1 给出了一种低效的 BPE 分词器训练实现，本质上就是逐步执行上文描述的流程。作为第一个练习，你可以先实现并测试该函数，以检查自己是否真正理解算法。

> **示例（`bpe_example`）：BPE 训练示例**
>
> 下面是改编自 R. Sennrich 等人 [3] 的一个示意性例子。假设语料库由以下文本组成：
>
> ```
> low low low low low
> lower lower widest widest widest
> newest newest newest newest newest newest
> ```
>
> 词表还包含 special token `<|endoftext|>`。
>
> **词表**
> 初始词表由 special token `<|endoftext|>` 和 256 个单字节值构成。
>
> **预分词**
> 为简化示例并突出 merge 过程，这里假设 pre-tokenization 只按空白字符切分。预分词并统计后得到频率表：
>
> ```
> {low: 5, lower: 2, widest: 3, newest: 6}
> ```
>
> 可以方便地把它表示为 `dict[tuple[bytes, ...], int]`，例如 `{(l,o,w): 5, …}`。注意，即使是单个字节，在 Python 中也仍然是一个 `bytes` 对象。Python 没有表示单一字节的 `byte` 类型，就像没有表示单一字符的 `char` 类型一样。
>
> **合并**
> 首先查看每一组相邻字节，并把它们在各单词中的出现频率相加，得到：
>
> ```
> {lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}
> ```
>
> `('e', 's')` 和 `('s', 't')` 频率并列，因此选择字典序更大的 `('s', 't')`。合并后得到：
>
> ```
> {(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}
> ```
>
> 第二轮中，`(e, st)` 是最常见的字节对（计数为 9），于是继续合并为：
>
> ```
> {(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6}
> ```
>
> 持续执行后，最终 merge 顺序为：
>
> ```
> ['s t', 'e st', 'o w', 'l ow', 'w est', 'n e',
>  'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']
> ```
>
> 如果只执行前 6 次 merge，则 merges 为：
>
> ```
> ['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']
> ```

> 此时词表元素是：
>
> `[<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]`
>
> 使用该词表和 merges，单词 `newest` 会被分词为 `[ne, west]`。

2.5 BPE 分词器训练实验

接下来在 TinyStories 数据集上训练字节级 BPE 分词器。数据集的查找和下载说明见第 1 节。开始前，建议先浏览 TinyStories 数据集，了解其中包含什么样的数据。

并行化 pre-tokenization

你会发现，pre-tokenization 是主要瓶颈之一。可以使用 Python 内置库 `multiprocessing` 对其并行化，以提升速度。

具体而言，在并行 pre-tokenization 实现中，建议把语料库分块，同时确保每个 chunk 的边界位于某个 special token 的起始位置。可以原样使用下列链接中的 starter code 来取得 chunk 边界，然后把不同 chunk 分配给不同进程：

`https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py`

这种分块方式始终有效，因为我们绝不希望跨文档边界进行 merge。就本作业而言，总可以按这种方式切分；无需考虑收到一个不包含 `<|endoftext|>` 的超大语料库这一边缘情况。

在 pre-tokenization 前移除 special tokens

使用正则模式运行 pre-tokenization（即使用 `re.finditer`）之前，应从语料库中移除全部 special token；如果是并行实现，则从每个 chunk 中移除。

必须按 special token 对文本进行切分，确保它们分隔的文本之间不会发生 merge。例如，若语料库（或 chunk）为：

`[Doc 1]<|endoftext|>[Doc 2]`

应当以 `<|endoftext|>` 为分隔符，将 `[Doc 1]` 和 `[Doc 2]` 分别进行 pre-tokenization，从而避免跨文档边界合并。

换言之，special token 在训练期间定义了不可跨越的硬分段边界，但它们自身不应参与 merge 计数。可以使用 `re.split` 完成这一点：用 `"|".join(special_tokens)` 作为分隔模式，并谨慎使用 `re.escape`，因为 special token 本身可能包含 `|`。测试 `test_train_bpe_special_tokens` 会检查这一行为。

优化 merge 步骤

上面示例中的朴素 BPE 训练实现很慢，因为每次 merge 都会遍历全部字节对以找出最高频字节对。然而，每次 merge 后，只有与被合并字节对发生重叠的那些 pair 的计数会改变。

因此，可以先索引全部 pair 的计数，并在每次 merge 后增量更新这些计数，而不是每轮都显式遍历所有字节对重新统计。使用这种缓存方案可获得显著加速。不过需要注意，BPE 训练中的 merge 部分在 Python 中无法并行化。

> **低资源提示：Profiling**
> 使用 `cProfile` 或 `py-spy` 等 profiling 工具找出实现中的瓶颈，并集中优化这些部分。

> **低资源提示：“Downscaling”**
> 不要一开始就在完整 TinyStories 数据集上训练分词器。建议先在一个较小的数据子集——即“debug dataset”——上训练。例如，可先使用 TinyStories validation set，它包含约 22K 篇文档，而完整训练集包含约 2.12M 篇文档。
> 这体现了一种通用开发策略：尽可能通过缩小数据集、模型规模等方式加速迭代。不过，debug dataset 的大小或超参数配置需要谨慎选择：它应足够大，以呈现完整配置中的同类瓶颈，使你的优化能够泛化；同时又不能大到每次运行都耗时过长。

题目（`train_bpe`）：BPE 分词器训练（15 分）

提交内容： 编写一个函数，给定输入文本文件路径，训练一个字节级 BPE 分词器。你的 BPE 训练函数至少应处理以下参数。

输入

- `input_path: str`：包含 BPE 分词器训练数据的文本文件路径。
- `vocab_size: int`：正整数，定义最终词表的最大大小；其中包括初始字节词表、merge 产生的词表项以及全部 special token。

- `special_tokens: list[str]`：要加入词表的字符串列表。训练期间，应把它们视为阻止跨越其范围进行 merge 的硬边界，但计算 merge 统计时不应把它们计入。

BPE 训练函数应返回得到的词表和 merges。

输出

- `vocab: dict[int, bytes]`：分词器词表，从 `int`（词表中的 token ID）映射到 `bytes`（token 字节）。
- `merges: list[tuple[bytes, bytes]]`：训练过程中产生的 BPE merges。每个列表元素是一个字节二元组 `(<token1>, <token2>)`，表示 `<token1>` 与 `<token2>` 被合并。列表应按 merge 创建顺序排列。

要使用课程组测试验证 BPE 训练函数，首先需要实现测试适配器 `[adapters.run_train_bpe]`，然后运行：

```
uv run pytest tests/test_train_bpe.py
```

你的实现应通过全部测试。

可选地（但这可能需要投入大量时间），可以使用系统编程语言实现训练方法的关键部分，例如 C++（可考虑 `cppyy` 或 `nanobind`）或 Rust（使用 PyO3）。如果这样做，请注意哪些操作需要复制数据、哪些操作可以直接读取 Python 内存，并留下构建说明，或确保只通过 `pyproject.toml` 即可完成构建。

还要注意，GPT-2 正则表达式在多数 regex engine 中支持不佳，而且在大多数引擎中速度过慢。课程组验证过 Oniguruma 速度尚可且支持 negative lookahead；不过 Python 的 `regex` 包通常甚至更快。

题目（`train_bpe_tinystories`）：在 TinyStories 上训练 BPE（2 分）

(a) 在 TinyStories 数据集上训练一个字节级 BPE 分词器，最大词表大小为 10,000。务必把 TinyStories 的 special token `<|endoftext|>` 加入词表。将最终词表和 merges 序列化到磁盘，以便进一步检查。训练耗时和内存占用是多少？词表中最长的 token 是什么？它合理吗？

资源要求： 不超过 30 分钟（不使用 GPU），不超过 30 GB RAM。

> **提示：** 在 pre-tokenization 阶段使用 `multiprocessing`，并利用以下两点，BPE 训练应能控制在 2 分钟以内：
> (a) `<|endoftext|>` token 在数据文件中分隔文档；
> (b) 在应用 BPE merges 前，`<|endoftext|>` 会作为 special case 处理。

提交内容： 一到两句话回答。

(b) 对代码进行 profiling。分词器训练流程中的哪个部分耗时最多？
提交内容： 一到两句话回答。

接下来尝试在 OpenWebText 数据集上训练字节级 BPE 分词器。与之前一样，建议先浏览数据集，以更好地理解其内容。

题目（`train_bpe_expts_owt`）：在 OpenWebText 上训练 BPE（2 分）

(a) 在 OpenWebText 数据集上训练一个字节级 BPE 分词器，最大词表大小为 32,000。将最终词表和 merges 序列化到磁盘，以便进一步检查。词表中最长的 token 是什么？它合理吗？

资源要求： 不超过 12 小时（不使用 GPU），不超过 100 GB RAM。
提交内容： 一到两句话回答。

(b) 比较在 TinyStories 和 OpenWebText 上训练得到的分词器，说明二者的异同。
提交内容： 一到两句话回答。

2.6 BPE 分词器：编码与解码

上一部分实现了一个函数：在输入文本上训练 BPE 分词器，从而得到词表和 BPE merge 列表。现在，我们将实现一个 BPE 分词器，它加载给定词表和 merges，并使用它们把文本编码为 token ID，或把 token ID 解码回文本。

2.6.1 编码文本

BPE 编码文本的过程与 BPE 词表的训练方式相呼应，主要包含以下步骤。

步骤 1：Pre-tokenize。 首先对输入序列进行预分词，并像 BPE 训练时一样，把每个 pre-token 表示为 UTF-8 字节序列。随后只在各 pre-token 内部把这些字节合并成词表元素；每个 pre-token 独立处理，不允许跨 pre-token 边界 merge。

步骤 2：应用 merges。 按照 BPE 训练时的创建顺序，把词表元素 merge 序列应用到各个 pre-token。

> **示例（`bpe_encoding`）：BPE 编码示例**
>
> 假设输入字符串为 `'the cat ate'`，词表为：
>
> ```
> {0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't',
>  6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}
> ```
>
> 学到的 merges 为：
>
> ```
> [(b't', b'h'), (b' ', b'c'), (b' ', b'a'),
>  (b'th', b'e'), (b' a', b't')]
> ```
>
> pre-tokenizer 首先把字符串切分为 `['the', ' cat', ' ate']`，然后分别对每个 pre-token 应用 BPE merges。

> 首个 pre-token `'the'` 初始表示为 `[b't', b'h', b'e']`。查看 merge 列表后，第一项可应用的 merge 是 `(b't', b'h')`，因此 pre-token 变为 `[b'th', b'e']`。随后重新检查 merges，下一项可应用的是 `(b'th', b'e')`，于是得到 `[b'the']`。此时整个 pre-token 已合并成单个 token，不再有可应用的 merge，因此对应整数序列为 `[9]`。
>
> 对剩余 pre-token 重复这一过程：`' cat'` 应用 merges 后表示为 `[b' c', b'a', b't']`，对应整数序列 `[7, 1, 5]`；最后的 `' ate'` 表示为 `[b' at', b'e']`，对应 `[10, 3]`。因此，整个输入字符串最终编码为：
>
> ```
> [9, 7, 1, 5, 10, 3]
> ```

Special tokens

分词器在编码文本时，应能正确处理用户定义的 special token；这些 token 会在构造分词器时提供。

内存注意事项

假设要对一个无法整体装入内存的大型文本文件进行分词。为了高效处理这个文件（或任何其他数据流），需要将它切分为可管理的 chunk，并依次处理，使内存复杂度保持常数级，而不是随文本大小线性增长。

切分时必须确保 token 不会跨 chunk 边界，否则得到的分词结果会与把整个序列一次性加载到内存中的朴素做法不同。

2.6.2 解码文本

要把整数 token ID 序列解码回原始文本，只需在词表中查找每个 ID 对应的字节序列，把这些字节连接起来，然后解码为 Unicode 字符串。

需要注意，输入 ID 不保证一定能映射为有效 Unicode 字符串，因为用户可能输入任意整数 ID 序列。如果输入 token ID 产生的字节串不是有效 Unicode，应使用 Unicode 官方替换字符 `U+FFFD` 替换格式错误的字节。`bytes.decode` 的 `errors` 参数控制如何处理 Unicode 解码错误；使用 `errors='replace'` 会自动把 malformed data 替换为该标记。[3]

题目（`tokenizer`）：实现分词器（15 分）

提交内容： 实现一个 `Tokenizer` 类。给定词表和 merge 列表，它应能把文本编码为整数 ID，并把整数 ID 解码回文本。分词器还应支持用户提供的 special token；如果这些 token 尚未在词表中，应将其追加到词表。建议采用以下接口。

```
def __init__(self, vocab, merges, special_tokens=None)
```

根据给定词表、merge 列表以及可选的 special token 列表构造分词器。参数：

```
vocab: dict[int, bytes]
merges: list[tuple[bytes, bytes]]
special_tokens: list[str] | None = None


@classmethod
def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)
```

类方法：从序列化后的词表文件和 merge 文件（格式与 BPE 训练代码的输出一致），以及可选的 special token 列表，构造并返回 `Tokenizer`。新增参数：

```
vocab_filepath: str
merges_filepath: str
special_tokens: list[str] | None = None
```

```
def encode(self, text: str) -> list[int]
```

把输入文本编码为 token ID 序列。

```
def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]
```

给定字符串 iterable（例如 Python 文件句柄），返回一个惰性产生 token ID 的 generator。该方法用于对无法整体加载到内存的大文件进行内存高效的分词。

```
def decode(self, ids: list[int]) -> str
```

把 token ID 序列解码为文本。

要使用课程组测试验证你的 `Tokenizer`，首先实现测试适配器 `[adapters.get_tokenizer]`，然后运行：

```
uv run pytest tests/test_tokenizer.py
```

你的实现应通过全部测试。

2.7 实验

题目（`tokenizer_experiments`）：分词器实验（4 分）

(a) 分别从 TinyStories 和 OpenWebText 中抽取 10 篇文档。使用先前训练的 TinyStories 和 OpenWebText 分词器（词表大小分别为 10K 和 32K），把这些样本文档编码为整数 ID。每个分词器的压缩率（bytes/token）是多少？
提交内容： 一到两句话回答。

(b) 如果使用 TinyStories 分词器对 OpenWebText 样本进行分词，会发生什么？比较压缩率，和/或定性描述结果。
提交内容： 一到两句话回答。

(c) 估计分词器吞吐量（例如 bytes/second）。对 Pile 数据集（825 GB 文本）完成分词大约需要多长时间？
提交内容： 一到两句话回答。

(d) 使用 TinyStories 和 OpenWebText 分词器，分别把对应的训练集和开发集编码为整数 token ID 序列。之后将使用这些数据训练语言模型。建议把 token ID 序列化为数据类型为 `uint16` 的 NumPy array。为什么 `uint16` 是合适的选择？
提交内容： 一到两句话回答。

3 Transformer 语言模型架构

语言模型接收一批整数 token ID 序列作为输入，即形状为 `(batch_size, sequence_length)` 的 `torch.Tensor`；输出则是词表上的一批归一化概率分布，即形状为 `(batch_size, sequence_length, vocab_size)` 的 PyTorch Tensor。对每个输入 token，输出分布预测其下一个 token。

训练语言模型时，使用这些 next-token prediction 计算真实下一个 token 与预测下一个 token 之间的交叉熵损失。推理生成文本时，取最后一个时间步（即序列最后一个位置）的预测分布来生成下一个 token，例如选择概率最高的 token，或从分布中采样；然后把生成的 token 加入输入序列，并重复该过程。

本部分将从零构建 Transformer 语言模型。我们先从高层描述开始，再逐步介绍各个组件。

3.1 Transformer LM

给定 token ID 序列，Transformer 语言模型首先通过 input embedding 把 token ID 转换为稠密向量；然后让嵌入后的 token 依次通过 `num_layers` 个 Transformer block；最后应用一个学习得到的线性投影（称为 output embedding 或 LM head），产生预测的 next-token logits。原文 Figure 1 给出了整体示意图。

Token Embeddings

第一步中，Transformer 把批量 token ID 序列嵌入为一组包含 token identity 信息的向量（原文 Figure 1 中的红色模块）。

更具体地说，token embedding layer 接收形状为 `(batch_size, sequence_length)` 的整数 tensor，输出形状为 `(batch_size, sequence_length, d_model)` 的向量序列。

Pre-norm Transformer Block

完成 embedding 后，activation 会经过若干结构相同的神经网络层。标准 decoder-only Transformer LM 由 `num_layers` 个相同层组成，这些层通常称为 Transformer “blocks”。

每个 Transformer block 接收形状为 `(batch_size, sequence_length, d_model)` 的输入，并返回同样形状的输出。每个 block 通过 self-attention 汇聚序列范围内的信息，并通过 feed-forward layer 对其进行非线性变换。

经过 `num_layers` 个 Transformer block 后，我们把最终 activation 转换成词表上的分布。

本作业实现 “pre-norm” Transformer block（详见第 3.4 节）。该架构还要求在最后一个 Transformer block 之后额外使用 layer normalization，以确保输出缩放合适。

归一化后，再使用标准的可学习线性变换，把 Transformer blocks 的输出转换为预测的 next-token logits（例如参见 A. Radford 等人 [7] 的 Equation 2）。

> **原文 Figure 1：Transformer 语言模型总览**
> 数据流依次为：Inputs → Token Embedding → `num_layers` 个 Transformer Blocks → Norm → Linear（Output Embedding）→ Softmax → Output Probabilities。
>
> **原文 Figure 2：Pre-norm Transformer block**
> 输入先经过 Norm，再进入带 RoPE 的 Causal Multi-Head Self-Attention，并通过 residual connection 相加；随后再次 Norm，进入 Position-Wise Feed-Forward，并再次通过 residual connection 相加。输入与输出形状均为 `(batch_size, seq_len, d_model)`。

3.2 说明：Batching、Einsum 与高效计算

在整个 Transformer 中，相同计算会应用于许多“类似 batch”的输入维度。例如：

- **batch 元素：** 对每个 batch element 应用相同的 Transformer forward 计算。
- **sequence length：** RMSNorm 和 feed-forward 等 position-wise 操作，在序列的每个位置上执行同样的计算。
- **attention heads：** attention 操作会在多个 head 上批量执行，形成 multi-head attention。

我们需要一种易于表达、能充分利用 GPU、同时代码容易阅读理解的实现方式。许多 PyTorch 操作允许 tensor 前端存在额外的“batch-like”维度，并能高效地在这些维度上重复或 broadcast 操作。

例如，假设执行 position-wise、batched operation。数据 tensor D 的形状为 `(batch_size, sequence_length, d_model)`，矩阵 A 的形状为 `(d_model, d_model)`，我们希望执行批量 vector-matrix multiply。此时 `D @ A` 会进行 batched matrix multiplication；PyTorch 会把 `(batch_size, sequence_length)` 视为 batch 维度，这是一个高效的基础操作。

因此，可以假设函数可能接收额外的 batch-like 维度，并把这些维度保留在 PyTorch tensor shape 的最前面。为了按这种形式组织 tensor，常常需要多次使用 `view`、`reshape` 和 `transpose`。这比较麻烦，也常常让代码含义和 tensor shape 变得难以阅读。

一种更方便的方案是在 `torch.einsum` 中使用 einsum notation，或更推荐使用与框架无关的库，如 `einops` 或 `einx`。两个核心操作是：

- `einsum`：可对输入 tensor 的任意维度进行 tensor contraction；
- `rearrange`：可对任意维度重新排序、拼接和拆分。

机器学习中的几乎所有操作，都可以视为维度变换与 tensor contraction 的某种组合，再偶尔加上通常为 pointwise 的非线性函数。因此，使用 einsum notation 可以让大量代码更易读、更灵活。

课程强烈建议学习并使用 einsum notation。此前没接触过 einsum 的同学应使用 `einops`（参阅其 docs）；已经熟悉 `einops` 的同学可以学习更通用的 `einx`。[4] 课程提供的环境已经安装了这两个包。

下面给出若干 einsum 用法示例，作为 `einops` 文档的补充；应先阅读 `einops` 文档。

> **示例（`einstein_example1`）：使用 `einops.einsum` 进行 batched matrix multiplication**

```
import torch
from einops import rearrange, einsum

## 基础实现
Y = D @ A.T
# 很难直接看出输入、输出 shape 及其语义。
# D 和 A 可以是什么 shape？其中是否存在意外行为？

## Einsum 具有自解释性且更加稳健
#                          D                A     ->          Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## 或者使用 batched 版本：D 可有任意 leading dimensions，而 A 受约束。
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

> **示例（`einstein_example2`）：使用 `einops.rearrange` 执行 broadcasted operations**
>
> 有一批图像，希望根据某个缩放因子，为每张图像生成 10 个不同程度的变暗版本：

```
images = torch.randn(64, 128, 128, 3)  # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## Reshape 后相乘
dim_value = rearrange(dim_by,    "dim_value              -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## 或一步完成
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

> **示例（`einstein_example3`）：使用 `einops.rearrange` 进行 pixel mixing**
>
> 假设一批图像表示为形状 `(batch, height, width, channel)` 的 tensor。我们希望对图像中的全部像素执行线性变换，但每个 channel 独立执行。线性变换由形状为 `(height * width, height * width)` 的矩阵 B 表示。

```
channels_last = torch.randn(64, 32, 32, 3)  # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## 重排图像 tensor，以便在所有像素之间进行 mixing
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)
```

使用 `einops` 可以写成：

```
height = width = 32

## rearrange 替代繁琐的 torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)
```

如果想更激进一些，可以使用 `einx.dot`（`einx` 中与 `einops.einsum` 对应的操作）一步完成：

```
height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)
```

第一个实现也可以在前后添加注释，说明输入输出 shape，从而改善可读性；但这种做法繁琐且容易出错。使用 einsum notation 时，**实现本身就是文档**。

Einsum notation 能处理任意输入 batch 维度，另一个关键优点是具有 self-documenting 性。使用 einsum 的代码中，输入和输出 tensor 的相关 shape 会清晰得多。对于其他 tensor，还可考虑使用 Tensor type hints，例如 `jaxtyping` 库（它并不局限于 JAX）。

作业 2 会进一步讨论使用 einsum notation 的性能影响；现阶段只需知道，在几乎所有情况下，它都比替代写法更好。

3.2.1 数学记号与内存排列顺序

许多机器学习论文使用行向量（row vector）记号，这种表示与 NumPy 和 PyTorch 默认采用的 row-major memory ordering 配合良好。使用行向量时，线性变换写作：

$$y = xW^\top. \tag{1}$$

其中，采用 row-major 存储的 $W \in \mathbb{R}^{d_{out} \times d_{in}}$，行向量 $x \in \mathbb{R}^{1 \times d_{in}}$。注意，这种写法可以通过增大 x 的最外层维度来批处理输入，也就是用矩阵输入 $X \in \mathbb{R}^{batch \times d_{in}}$ 替代向量输入 x。

在线性代数中，更常见的是列向量（column vector）记号，此时线性变换写作：

$$y = Wx. \tag{2}$$

这里，$W \in \mathbb{R}^{d_{out} \times d_{in}}$ 仍采用 row-major 存储，列向量 $x \in \mathbb{R}^{d_{in}}$。若要在这种约定下对输入进行 batch，$x$ 的 batch dimension 必须放在最后，因此要把 x 替换为矩阵 $\tilde{X} \in \mathbb{R}^{d_{in} \times batch}$。

本作业的数学记号主要采用列向量，因为数学文献通常遵循这一约定。需要牢记：PyTorch 使用 row-major memory ordering，因此若在代码中直接使用普通矩阵乘法记号，应用矩阵时通常需要像 Equation 1 的行向量约定那样取 transpose。若使用 einsum 进行线性代数运算，只要轴标签正确，这个问题基本可以忽略。

顺带一提，Matlab、Julia、Fortran 等语言或线性代数包采用 column-major memory ordering，因此 batch dimension 通常放在最后；Python 及其相关包则采用 C 标准的 row-major ordering。

3.3 基本构件：Linear 与 Embedding 模块

3.3.1 参数初始化

要有效训练神经网络，通常需要谨慎初始化模型参数。糟糕的初始化可能导致梯度消失、梯度爆炸等不良行为。Pre-norm Transformer 对初始化异常稳健，但初始化仍可能显著影响训练速度和收敛表现。

由于本作业已经很长，初始化细节留到作业 3 再讨论。现阶段使用以下近似初始化方案，它们在大多数情况下都应工作良好：

- **Linear 权重：** $\mathcal{N}(\mu = 0, \sigma^2 = \frac{2}{d_{in}+d_{out}})$，截断到 $[-3\sigma, 3\sigma]$。
- **Embedding：** $\mathcal{N}(\mu = 0, \sigma^2 = 1)$，截断到 $[-3, 3]$。
- **RMSNorm：** 初始化为全 1 向量 $\mathbf{1}$。

应使用 `torch.nn.init.trunc_normal_` 初始化截断正态分布权重。

3.3.2 Linear 模块

Linear layer 是 Transformer 以及一般神经网络的基础构件。首先实现自定义 `Linear` 类，它继承 `torch.nn.Module` 并执行线性变换：

$$y = Wx. \tag{3}$$

遵循多数现代 LLM 的做法，这里不包含 bias term。

题目（`linear`）：实现 Linear 模块（1 分）

提交内容： 实现一个继承 `torch.nn.Module` 的 `Linear` 类，执行线性变换。除不提供 bias 参数或 bias parameter 外，接口应与 PyTorch 内置的 `nn.Linear` 相同。建议接口如下：

```
def __init__(self, in_features, out_features, device=None, dtype=None)
```

构造线性变换模块。参数：

- `in_features: int`： 输入的最后一个维度。
- `out_features: int`： 输出的最后一个维度。
- `device: torch.device | None = None`： 参数存储设备。
- `dtype: torch.dtype | None = None`： 参数数据类型。

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

对输入应用线性变换。

务必做到：

- 继承 `nn.Module`；
- 调用父类构造函数；
- 按 W 而不是 $W^\top$ 的形式构造并存储参数，并将其放入 `nn.Parameter`；
- 不使用 `nn.Linear` 或 `nn.functional.linear`。

初始化时使用上一节给出的设置，并使用 `torch.nn.init.trunc_normal_` 初始化权重。

要测试 `Linear` 模块，实现测试适配器 `[adapters.run_linear]`。适配器应把给定权重加载到你的 `Linear` 模块中；可使用 `Module.load_state_dict`。然后运行：

```
uv run pytest -k test_linear
```

3.3.3 Embedding 模块

如前所述，Transformer 的第一层是 embedding layer，它把整数 token ID 映射到维度为 `d_model` 的向量空间。

我们将实现一个继承 `torch.nn.Module` 的自定义 `Embedding` 类，因此不能使用 `nn.Embedding`。 `forward` 方法应使用形状为 `(batch_size, sequence_length)` 的 token ID `torch.LongTensor`，索引形状为 `(vocab_size, d_model)` 的 embedding matrix，为每个 token ID 选择对应 embedding vector。

题目（`embedding`）：实现 Embedding 模块（1 分）

提交内容： 实现继承 `torch.nn.Module` 的 `Embedding` 类，完成 embedding lookup。接口应与 PyTorch 内置的 `nn.Embedding` 一致。建议接口如下：

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)
```

构造 embedding 模块。参数：

- `num_embeddings: int`： 词表大小。
- `embedding_dim: int`： embedding vector 维度，即 d_{model}。
- `device: torch.device | None = None`： 参数存储设备。
- `dtype: torch.dtype | None = None`： 参数数据类型。

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor
```

查找给定 token ID 对应的 embedding vector。

务必做到：

- 继承 `nn.Module`；
- 调用父类构造函数；
- 把 embedding matrix 初始化为 `nn.Parameter`；
- 存储 embedding matrix 时，使 `d_model` 为最后一个维度；
- 不使用 `nn.Embedding` 或 `nn.functional.embedding`。

同样使用上一节设置进行初始化，并使用 `torch.nn.init.trunc_normal_` 初始化权重。

实现测试适配器 `[adapters.run_embedding]` 后，运行：

```
uv run pytest -k test_embedding
```

3.4 Pre-Norm Transformer Block

每个 Transformer block 包含两个 sub-layer：multi-head self-attention mechanism 和 position-wise feed-forward network（见 [A. Vaswani et al., 2017] 第 3.1 节）。

原始 Transformer 论文在两个 sub-layer 外各使用 residual connection，然后应用 layer normalization。由于 layer normalization 施加在 sub-layer 输出上，这种架构通常称为 “post-norm” Transformer。

然而，多项工作发现，把 layer normalization 从每个 sub-layer 的输出移动到输入，并在最后一个 Transformer block 后再添加一次 layer normalization，能提高 Transformer 训练稳定性 [T. Q. Nguyen et al., 2019; R. Xiong et al., 2020]。原文 Figure 2 展示了这种 “pre-norm” Transformer block。

每个 Transformer block sub-layer 的输出通过 residual connection 加回 sub-layer 输入（A. Vaswani et al. [8] 第 5.4 节）。对 pre-norm 的一种直觉是：从 input embedding 到 Transformer 最终输出之间，存在一条不经过归一化的干净 “residual stream”，据称这能改善 gradient flow。

Pre-norm Transformer 已成为当今语言模型（例如 GPT-3、LLaMA、PaLM 等）的标准选择，因此本作业实现这一变体。下面按顺序介绍并实现 pre-norm Transformer block 的各个组件。

3.4.1 Root Mean Square Layer Normalization

A. Vaswani 等人 [8] 的原始 Transformer 使用 layer normalization [J. L. Ba et al., 2016] 归一化 activation。遵循 H. Touvron 等人 [12]，本作业使用 root mean square layer normalization（RMSNorm；B. Zhang et al. [13] Equation 4）。

给定 activation 向量 $a \in \mathbb{R}^{d_{model}}$，RMSNorm 按如下方式缩放每个 activation a_i：

$$\mathrm{RMSNorm}(a_i) = \frac{a_i}{\mathrm{RMS}(a)} g_i, \tag{4}$$

其中：

$$\mathrm{RMS}(a) = \sqrt{\frac{1}{d_{model}} \sum_{i=1}^{d_{model}} a_i^2 + \varepsilon}.$$

g_i 是可学习的 gain parameter，总共有 `d_model` 个；ε 是超参数，通常固定为 `1e-5`。

对输入平方时，为防止溢出，应先将输入 upcast 为 `torch.float32`。整体 `forward` 方法应类似：

```
in_dtype = x.dtype
x = x.to(torch.float32)

# 在这里实现 RMSNorm
...
result = ...

# 使用原始 dtype 返回结果
return result.to(in_dtype)
```

题目（ `rmsnorm` ）：Root Mean Square Layer Normalization（1 分）

提交内容： 将 RMSNorm 实现为一个 `torch.nn.Module` 。建议接口：

```
def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)
```

构造 RMSNorm 模块。参数：

- `d_model: int` ： 模型 hidden dimension。
- `eps: float = 1e-5` ： 用于 numerical stability 的 epsilon。
- `device: torch.device | None = None` ： 参数存储设备。
- `dtype: torch.dtype | None = None` ： 参数数据类型。

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

处理形状为 `(batch_size, sequence_length, d_model)` 的输入 tensor，返回相同形状的 tensor。

注意：归一化前应按上文要求把输入 upcast 为 `torch.float32` ，之后再 downcast 回原始 dtype。

实现测试适配器 `[adapters.run_rmsnorm]` 后，运行：

```
uv run pytest -k test_rmsnorm
```

3.4.2 Position-Wise Feed-Forward Network

原始 Transformer 论文（A. Vaswani 等人 [8] 第 3.3 节）中的 feed-forward network 由两个线性变换组成，中间使用 ReLU activation：

$$\mathrm{ReLU}(x) = \max(0, x).$$

在原始架构中，内部 feed-forward layer 的维度通常是输入维度的 4 倍。

与原始设计相比，现代语言模型通常有两个主要变化：使用不同的 activation function，并加入 gating mechanism。具体来说，本作业将实现 Llama 3 [A. Grattafiori et al., 2024]、Qwen 2.5 [A. Yang et al., 2024] 等 LLM 采用的 “SwiGLU” activation。它把 SiLU（也常称为 Swish）activation 与一种称为 Gated Linear Unit（GLU）的 gating mechanism 结合起来。

同时，遵循自 PaLM [A. Chowdhery et al., 2022] 和 LLaMA [H. Touvron et al., 2023] 以来多数现代 LLM 的做法，我们会省略 linear layer 有时使用的 bias term。

SiLU/Swish activation function [D. Hendrycks et al., 2016; S. Elfwing et al., 2017] 定义为：

$$\mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}. \tag{5}$$

从原文 Figure 3 可以看出，SiLU 与 ReLU 类似，但在零点处是光滑的。

> **原文 Figure 3：SiLU（又称 Swish）与 ReLU activation function 的比较。**
> 图中同时绘制了 Identity、ReLU 和 SiLU。SiLU 在负半轴可取略小于 0 的值，并在 0 附近平滑过渡。

Gated Linear Unit（GLU）最初由 Y. N. Dauphin 等人 [19] 定义为：一个经过 sigmoid 的线性变换与另一个线性变换的逐元素乘积：

$$\mathrm{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \tag{6}$$

其中 $\odot$ 表示 element-wise multiplication。GLU 被认为能够“为梯度提供一条线性路径，同时保留非线性能力，从而缓解深层架构中的 vanishing gradient 问题”。

把 SiLU/Swish 与 GLU 结合起来，即得到本作业 feed-forward network 使用的 SwiGLU：

$$\mathrm{FFN}(x) = \mathrm{SwiGLU}(x, W_1, W_2, W_3) = W_2\left(\mathrm{SiLU}(W_1 x) \odot W_3 x\right). \tag{7}$$

其中：

- $x \in \mathbb{R}^{d_{model}}$；
- $W_1, W_3 \in \mathbb{R}^{d_{ff} \times d_{model}}$；
- $W_2 \in \mathbb{R}^{d_{model} \times d_{ff}}$；
- 标准取值为 $d_{ff} = \frac{8}{3} d_{model}$。

具体实现时，为提高硬件效率，可以把它舍入到附近的 64 的倍数。

N. Shazeer [20] 首先提出把 SiLU/Swish 与 GLU 结合，并通过实验表明，在语言建模任务上，SwiGLU 优于 ReLU 和不带 gating 的 SiLU 等 baseline。本作业后续将比较 SwiGLU 与 SiLU。

尽管上文给出了一些 heuristic argument，相关论文也提供了更多证据，但仍应保持 empirical perspective。Shazeer 论文中有一句著名的话：

> “我们无法解释为什么这些架构似乎有效；与其他一切一样，我们把它们的成功归因于神的恩典。”

题目（`positionwise_feedforward`）：实现 position-wise feed-forward network（2 分）

提交内容： 实现由 SiLU activation 和 GLU 组成的 SwiGLU feed-forward network。

注意：在这里可以使用 `torch.sigmoid`，以提高 numerical stability。

实现中应将 d_{ff} 设置为约 $\frac{8}{3} d_{model}$，同时确保内部 feed-forward layer 的维度为 64 的倍数，以充分利用硬件。

实现测试适配器 `[adapters.run_swiglu]` 后，运行：

```
uv run pytest -k test_swiglu
```

3.4.3 Relative Positional Embeddings

为了向模型注入位置信息，本作业实现 Rotary Position Embeddings [J. Su et al., 2021]，通常简称 RoPE。

对于 token 位置 i 上的 query token：

$$q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d,$$

应用成对旋转矩阵 R_i，得到：

$$q'^{(i)} = R_i q^{(i)} = R_i W_q x^{(i)}.$$

R_i 会把 embedding element 的每一对 $q^{(i)}_{2k-1:2k}$ 当作二维向量，旋转角度为：

$$\theta_{i,k} = \frac{i}{\Theta^{(2k-2)/d}}, \qquad k \in \{1, \ldots, d/2\},$$

其中 Θ 是常数。

因此，可以把 R_i 看作一个 $d \times d$ 的 block-diagonal matrix，其中第 k 个 2×2 block 为：

$$R^i_k = \begin{pmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{pmatrix}. \tag{8}$$

完整旋转矩阵为：

$$R^i = \begin{pmatrix} R^i_1 & 0 & 0 & \cdots & 0 \\ 0 & R^i_2 & 0 & \cdots & 0 \\ 0 & 0 & R^i_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & R^i_{d/2} \end{pmatrix}, \tag{9}$$

其中的 0 表示 2×2 零矩阵。

虽然可以显式构造完整的 $d \times d$ 矩阵，但好的实现应利用该矩阵的结构，更高效地完成变换。

由于我们只关心同一序列内 token 之间的相对旋转，计算得到的 $\cos(\theta_{i,k})$ 和 $\sin(\theta_{i,k})$ 可以跨 layer、跨 batch 复用。若要优化，可让全部 layer 共享一个 RoPE 模块，并在 `__init__` 中用 `self.register_buffer(persistent=False)` 创建预计算的二维 sin/cos buffer，而不是使用 `nn.Parameter`，因为这些固定的 sine/cosine 值不应被学习。

对 key $k^{(j)}$ 执行与 query 相同的旋转过程，使用相应的 R_j。注意，该 layer 没有 learnable parameter。

题目（`rope`）：实现 RoPE（2 分）

提交内容： 实现 `RotaryPositionalEmbedding` 类，对输入 tensor 应用 RoPE。

建议接口：

```
def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)
```

构造 RoPE 模块，并按需创建 buffer。参数：

- `theta: float`：RoPE 的 Θ 值。
- `d_k: int`：query 和 key vector 的维度。
- `max_seq_len: int`：可能输入的最大 sequence length。
- `device: torch.device | None = None`：buffer 存储设备。

```
def forward(
    self,
    x: torch.Tensor,
    token_positions: torch.Tensor,
) -> torch.Tensor
```

处理形状为 `(..., seq_len, d_k)` 的输入 tensor，并返回相同形状的 tensor。应允许 x 具有任意数量的 batch dimensions。

假设 `token_positions` 的形状为 `(..., seq_len)`，指定 x 在 sequence dimension 上对应的 token positions。应使用这些 token position 沿 sequence dimension 索引或切片可能已预计算的 cos/sin tensor。

完成 `[adapters.run_rope]` 后，确保通过：

```
uv run pytest -k test_rope
```

3.4.4 Scaled Dot-Product Attention

现在实现 A. Vaswani 等人 [8] 第 3.2.1 节描述的 scaled dot-product attention。

作为预备，Attention 定义会使用 softmax。Softmax 把未经归一化的 score vector 转换为归一化分布：

$$\mathrm{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^{n} \exp(v_j)}. \tag{10}$$

当 v_i 很大时，$\exp(v_i)$ 可能变成 `inf`，随后产生 `inf / inf = NaN`。可以利用 softmax 的一个性质避免这个问题：给所有输入同时加任意常数 c，结果不变。

为了 numerical stability，通常从 v 的每个元素中减去最大值，使新的最大元素为 0。接下来使用这一技巧实现 softmax。

题目（`softmax`）：实现 softmax（1 分）

提交内容： 编写函数，对 tensor 应用 softmax。函数接收两个参数：一个 tensor 和一个维度 i，并沿输入 tensor 的第 i 个维度应用 softmax。

输出 tensor 的 shape 应与输入相同，但第 i 个维度现在表示归一化概率分布。为避免 numerical stability 问题，应从第 i 个维度的所有元素中减去该维度上的最大值。

完成 `[adapters.run_softmax]` 后，确保通过：

```
uv run pytest -k test_softmax_matches_pytorch
```

Attention operation 可定义为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{11}$$

其中：

- $Q \in \mathbb{R}^{n \times d_k}$；
- $K \in \mathbb{R}^{m \times d_k}$；
- $V \in \mathbb{R}^{m \times d_v}$。

这里的 Q、K、V 都是该 operation 的输入，不是 learnable parameter。

Masking

有时需要 mask attention operation 的输出。mask 的形状应为：

$$M \in \{\text{True}, \text{False}\}^{n \times m}.$$

该 boolean matrix 的第 i 行表示 query i 应关注哪些 key。按照惯例（虽然容易混淆），位置 (i, j) 为 `True` 表示 query i **可以** attend to key j； `False` 表示不能。也就是说，“信息流”通过值为 `True` 的 (i, j) pair。

例如，一个 `1 × 3` 的 mask：

```
[[True, True, False]]
```

表示唯一的 query vector 只关注前两个 key。

计算上，使用 masking 比分别对子序列计算 attention 高效得多。具体做法是在 softmax 之前的值：

$$\frac{QK^\top}{\sqrt{d_k}}$$

中，对 mask 为 `False` 的位置加上 $-\infty$。

题目（`scaled_dot_product_attention`）：实现 scaled dot-product attention（5 分）

提交内容： 实现 scaled dot-product attention 函数。

实现应支持：

- key 和 query shape：`(batch_size, ..., seq_len, d_k)`；
- value shape：`(batch_size, ..., seq_len, d_v)`；
- 其中 `...` 表示任意数量的其他 batch-like dimension；
- 返回 shape：`(batch_size, ..., seq_len, d_v)`。

关于 batch-like dimension，参见第 3.2 节。

实现还应支持可选的用户提供 boolean mask，shape 为 `(seq_len, seq_len)`。mask 为 `True` 的位置对应的 attention probability 总和应为 1，mask 为 `False` 的位置对应概率应为 0。

实现测试适配器 `[adapters.run_scaled_dot_product_attention]` 后：

- `uv run pytest -k test_scaled_dot_product_attention` 测试三阶输入 tensor；
- `uv run pytest -k test_4d_scaled_dot_product_attention` 测试四阶输入 tensor。

3.4.5 Causal Multi-Head Self-Attention

按照 A. Vaswani 等人 [8] 第 3.2.2 节实现 multi-head self-attention。数学上，multi-head attention 定义为：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h), \tag{12}$$

其中：

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i). \tag{13}$$

Q_i、K_i、V_i 分别是 Q、K、V embedding dimension 上第 $i \in \{1, \ldots, h\}$ 个大小为 d_k 或 d_v 的切片。这里的 `Attention` 就是第 3.4.4 节定义的 scaled dot-product attention。

由此得到 multi-head self-attention：

$$\text{MultiHeadSelfAttention}(x) = W_O \,\text{MultiHead}(W_Q x, W_K x, W_V x). \tag{14}$$

learnable parameters 为：

- $W_Q \in \mathbb{R}^{hd_k \times d_{model}}$；
- $W_K \in \mathbb{R}^{hd_k \times d_{model}}$；
- $W_V \in \mathbb{R}^{hd_v \times d_{model}}$；
- $W_O \in \mathbb{R}^{d_{model} \times hd_v}$。

由于在 multi-head attention 中会切分 Q、K、V，可把 W_Q、W_K、W_V 理解为沿输出维度按 head 分开。正确实现后，key、value、query projection 总共应只执行三次 matrix multiplication。[5]

Causal masking

实现必须阻止模型关注序列中的未来 token。

例如，输入 token 序列为 $t_1, \ldots, t_n$，当计算前缀 $t_1, \ldots, t_i$（$i < n$）的 next-word prediction 时，模型不得访问位置 $t_{i+1}, \ldots, t_n$ 的 token representation。因为推理生成文本时这些未来 token 尚不存在；而它们还会泄漏真实下一个词的信息，使语言模型 pre-training objective 变得过于简单。

朴素做法是对序列的 n 个不同 prefix 分别运行一次 multi-head self-attention，共运行 n 次。更高效的方法是 causal attention mask：允许 token i attend to 序列中所有满足 $j \le i$ 的位置。

可使用 `torch.triu` 或 broadcasted index comparison 构造该 mask，并复用第 3.4.4 节中已支持 attention masking 的 scaled dot-product attention 实现。

应用 RoPE

RoPE 应应用到 query 和 key vector，而不能应用到 value vector。

同时，head dimension 应作为 batch dimension 处理，因为 multi-head attention 中每个 head 独立执行 attention。这意味着，对每个 head 的 query/key vector 应应用完全相同的 RoPE rotation。

题目（`multihead_self_attention`）：实现 causal multi-head self-attention（5 分）

提交内容： 将 causal multi-head self-attention 实现为一个 `torch.nn.Module`。实现至少应接收以下参数：

- `d_model: int`：Transformer block 输入的维度。
- `num_heads: int`：multi-head self-attention 使用的 head 数量。

遵循 A. Vaswani 等人 [8]，设置：

$$d_k = d_v = \frac{d_{model}}{h}.$$

实现测试适配器 `[adapters.run_multihead_self_attention]` 后，运行：

```
uv run pytest -k test_multihead_self_attention
```

3.5 完整 Transformer LM

先组装 Transformer block（可回看原文 Figure 2）。一个 Transformer block 包含两个 sub-layer：一个用于 multi-head self-attention，另一个用于 SwiGLU feed-forward network。

在每个 sub-layer 中，先执行 RMSNorm，再执行主要操作（MHA/FF），最后加入 residual connection。

具体而言，Transformer block 的前半部分（第一个 sub-layer）应根据输入 x 计算输出 y：

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \tag{15}$$

题目（`transformer_block`）：实现 Transformer block（3 分）

按照第 3.4 节的说明和原文 Figure 2，实现 pre-norm Transformer block。至少应接收以下参数：

- `d_model: int`：Transformer block 输入维度。
- `num_heads: int`：multi-head self-attention 使用的 head 数量。
- `d_ff: int`：position-wise feed-forward inner layer 的维度。

实现适配器 `[adapters.run_transformer_block]`，然后运行：

```
uv run pytest -k test_transformer_block
```

提交内容： 能通过课程组测试的 Transformer block 代码。

现在根据原文 Figure 1 的高层结构把各个 block 组合起来。按照第 3.1 节对 embedding 的描述，将 embedding 输出送入 `num_layers` 个 Transformer block，再通过最终 layer norm 和 LM head，得到词表上的未归一化分布，即 logits。

题目（`transformer_lm`）：实现 Transformer LM（3 分）

现在把全部组件组合起来。按照第 3.1 节和原文 Figure 1 实现 Transformer language model。

至少应接受前文 Transformer block 的全部构造参数，并额外接受：

- `vocab_size: int`：词表大小，用于确定 token embedding matrix 的维度。
- `context_length: int`：最大 context length，用于确定 RoPE sin/cos buffer 的维度。
- `num_layers: int`：使用的 Transformer block 数量。

首先实现测试适配器 `[adapters.run_transformer_lm]`，然后运行：

```
uv run pytest -k test_transformer_lm
```

提交内容： 能通过上述测试的 Transformer LM module。

资源核算（Resource accounting）

理解 Transformer 各部分如何消耗计算和内存十分有用。下面做基本的 FLOPs accounting。

Transformer 中绝大多数 FLOPs 来自 matrix multiplication，因此核心方法很简单：

1. 列出一次 Transformer forward pass 中的全部 matrix multiplication。
2. 把每次 matrix multiplication 换算成所需 FLOPs。

第二步会用到以下事实：

> **规则：** 给定 $A \in \mathbb{R}^{m \times n}$ 和 $B \in \mathbb{R}^{n \times p}$，矩阵乘积 AB 需要 $2mnp$ FLOPs。

原因是：

$$(AB)[i, j] = A[i, :] \cdot B[:, j],$$

这个 dot product 需要 n 次加法和 n 次乘法，共 $2n$ FLOPs。矩阵 AB 有 $m \times p$ 个元素，因此总 FLOPs 为：

$$(2n)(mp) = 2mnp.$$

在做下一题前，建议逐一检查 Transformer block 和 Transformer LM 的各个组件，列出其中所有 matrix multiplication 及对应 FLOPs cost。

题目（`transformer_accounting`）：Transformer LM 资源核算（5 分）

(a) 考虑一个采用本作业架构、规模与 GPT-2 XL 相当的模型，配置如下：

```
vocab_size: 50,257
context_length: 1,024
num_layers: 48
d_model: 1,600
num_heads: 25
d_ff: 4,288  # 最接近 (8/3) × 1,600 的 64 倍数
```

若按该配置构造模型，可训练参数共有多少？假设每个参数都使用 single-precision floating point 表示，仅加载模型需要多少内存？
提交内容： 一到两句话回答。

(b) 找出完成该 GPT-2 XL 规模模型一次 forward pass 所需的 matrix multiplication。它们总共需要多少 FLOPs？假设输入序列包含 `context_length` 个 token。
提交内容： matrix multiplication 列表（含说明），以及总 FLOPs。

(c) 根据上述分析，模型的哪些部分消耗 FLOPs 最多？
提交内容： 一到两句话回答。

(d) 分别对 GPT-2 small（12 层、`d_model=768`、12 heads）、GPT-2 medium（24 层、`d_model=1024`、16 heads）和 GPT-2 large（36 层、`d_model=1280`、20 heads）重复分析。随着模型增大，Transformer LM 各部分在总 FLOPs 中所占比例如何增加或减少？
提交内容： 对每个模型给出各组件 FLOPs 及其占总 forward pass FLOPs 的比例；另用一到两句话说明模型规模变化如何改变各组件的 FLOPs 比例。

(e) 对 GPT-2 XL，将 context length 增加到 16,384。一次 forward pass 的总 FLOPs 如何变化？各模型组件的 FLOPs 相对贡献如何变化？
提交内容： 一到两句话回答。

4 训练 Transformer LM

现在已经有了数据预处理步骤（tokenizer）和模型（Transformer）。剩下的是构建支持训练的全部代码，包括：

- **Loss：** 定义 loss function（cross-entropy）。
- **Optimizer：** 定义用于最小化 loss 的 optimizer（AdamW）。
- **Training loop：** 构建加载数据、保存 checkpoint、管理训练的基础设施。

4.1 Cross-Entropy Loss

回顾一下：对于任意长度为 $m+1$ 的序列 x，以及 $i = 1, \dots, m$，Transformer LM 定义分布：

$$p_\theta(x_{i+1} \mid x_{1:i}).$$

给定由长度为 $m+1$ 的序列组成的训练集 D，标准 cross-entropy（negative log-likelihood）loss 定义为：

$$\ell(\theta; D) = \frac{1}{|D|m} \sum_{x \in D} \sum_{i=1}^{m} -\log p_\theta(x_{i+1} \mid x_{1:i}). \tag{16}$$

注意，一次 Transformer forward pass 会同时产生所有 $i = 1, \dots, m$ 的 $p_\theta(x_{i+1} \mid x_{1:i})$。

具体而言，Transformer 在每个位置 i 计算 logits $o_i \in \mathbb{R}^{vocab_size}$，因此：[6]

$$p(x_{i+1} \mid x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{vocab_size} \exp(o_i[a])}. \tag{17}$$

cross-entropy loss 通常相对于 logits vector $o_i \in \mathbb{R}^{vocab_size}$ 和 target x_{i+1} 定义。[7]

与 softmax 一样，实现 cross-entropy loss 时也必须谨慎处理 numerical issue。

题目（`cross_entropy`）：实现 cross-entropy（1 分）

提交内容： 编写函数计算 cross-entropy loss。输入为预测 logits o_i 和 target x_{i+1}，计算：

$$\ell_i = -\log \text{softmax}(o_i)[x_{i+1}].$$

函数应满足：

- 为 numerical stability，减去最大元素；
- 尽可能消去 `log` 与 `exp`；
- 支持任意额外 batch dimension，并返回 batch 上的平均值。与第 3.2 节一致，假设 batch-like dimension 总在前面，vocabulary size dimension 位于最后。

实现 `[adapters.run_cross_entropy]`，然后运行：

```
uv run pytest -k test_cross_entropy
```

Perplexity

Cross-entropy 足以用于训练，但评估模型时还希望报告 perplexity。对于长度为 m、各位置 cross-entropy loss 为 $\ell_1, \dots, \ell_m$ 的序列：

$$\text{perplexity} = \exp\left(\frac{1}{m} \sum_{i=1}^{m} \ell_i\right). \tag{18}$$

4.2 SGD 优化器

有了 loss function 后，开始研究 optimizer。最简单的 gradient-based optimizer 是 Stochastic Gradient Descent（SGD）。

从随机初始化参数 θ_0 开始，对每个 step $t = 0, \ldots, T-1$，执行：

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \tag{19}$$

其中 B_t 是从数据集 D 随机采样的 batch，learning rate α_t 和 batch size $|B_t|$ 是 hyperparameter。

4.2.1 在 PyTorch 中实现 SGD

为了实现 optimizer，我们将继承 PyTorch 的 `torch.optim.Optimizer`。一个 `Optimizer` 子类必须实现两个方法。

```
def __init__(self, params, ...)
```

初始化 optimizer。`params` 是待优化参数的集合；如果用户希望模型不同部分使用不同 hyperparameter（例如不同 learning rate），也可能传入 parameter groups。

务必把 `params` 传给基类的 `__init__`，基类会存储这些参数，以便 `step` 使用。你可以根据 optimizer 接收额外参数（例如常见的 learning rate），并把它们以 dictionary 的形式传给基类构造函数；dictionary 的 key 是你为这些参数选择的字符串名称。

```
def step(self)
```

执行一次参数更新。training loop 中会在 backward pass 后调用它，因此此时可以访问最近一个 batch 产生的 gradient。

该方法应遍历每个 parameter tensor `p`，并基于其 gradient `p.grad`（若存在）原地修改 `p.data`。`p.data` 保存与参数关联的 tensor，而 `p.grad` 表示 loss 对该参数的 gradient tensor。

PyTorch optimizer API 有一些细节，用示例更容易解释。下面实现 SGD 的一个轻微变体：learning rate 随训练衰减，从初始 learning rate α 开始，步长逐渐减小：

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}} \nabla L(\theta_t; B_t). \tag{20}$$

PyTorch `Optimizer` 实现如下：

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"]  # 获取 learning rate。
            for p in group["params"]:
                if p.grad is None:
                    continue

                state = self.state[p]       # 获取与 p 关联的 state。
                t = state.get("t", 0)      # 从 state 中取迭代次数，默认为 0。
                grad = p.grad.data          # 获取 loss 对 p 的 gradient。
                p.data -= lr / math.sqrt(t + 1) * grad  # 原地更新权重 tensor。
                state["t"] = t + 1         # 迭代次数加 1。

        return loss
```

在 `__init__` 中，把参数和默认 hyperparameter 传给基类构造函数。参数可能被组织成多个 group，每组有不同 hyperparameter。若参数只是单个 `torch.nn.Parameter` 集合，基类构造函数会创建一个 parameter group，并为其分配默认 hyperparameter。

在 `step` 中，先遍历每个 parameter group，再遍历该组的每个 parameter，并应用 Equation 20。迭代次数作为每个 parameter 关联的 state 保存：先读取该值，用于 gradient update，再更新它。

API 规定用户可能传入 callable `closure`，在 optimizer step 前重新计算 loss。本作业使用的 optimizer 不需要它，但为了符合 API，示例仍加入该参数。

下面是一个最小 training loop 示例：

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad()               # 重置所有 learnable parameter 的 gradient。
    loss = (weights**2).mean()    # 计算 scalar loss。
    print(loss.cpu().item())


    loss.backward()  # 运行 backward pass，计算 gradient。
    opt.step()       # 执行 optimizer step。
```

这就是典型 training loop 的结构：每次迭代计算 loss，然后执行一次 optimizer step。训练语言模型时，learnable parameters 来自模型；在 PyTorch 中，`m.parameters()` 会返回这一参数集合。loss 在采样得到的数据 batch 上计算，但 training loop 的基本结构不变。

题目（`learning_rate_tuning`）：调整 learning rate（1 分）

如后文所示，learning rate 是对训练影响最大的 hyperparameter 之一。现在在上述 toy example 中实际观察这一点。

分别把 learning rate 改为 `1e1`、`1e2` 和 `1e3`，每种只运行 10 次 training iteration。每个 learning rate 下 loss 会发生什么？它衰减得更快、更慢，还是会 diverge（即训练过程中持续增大）？

提交内容： 用一到两句话说明观察到的行为。

4.3 AdamW

现代语言模型通常不使用 SGD，而使用更复杂的 optimizer。近年来最常见的 optimizer 大多是 Adam [D. P. Kingma et al., 2015] 的派生方法。本作业使用在近期工作中广泛采用的 AdamW [I. Loshchilov et al., 2019]。

AdamW 对 Adam 做出修改：以与 gradient update 解耦的方式加入 weight decay——每次迭代都把参数向 0 拉近——从而改善 regularization。本作业按 I. Loshchilov 等人 [23] 的 Algorithm 2 实现 AdamW。

AdamW 是 stateful optimizer：它会为每个 parameter 维护一阶矩和二阶矩的 running estimate。因此，它以额外内存为代价换取更好的稳定性和收敛表现。

除 learning rate α 外，AdamW 还包含：

- 一对控制 moment estimate 更新的 hyperparameter (β_1, β_2)；
- weight decay rate λ。

典型应用会把 (β_1, β_2) 设为 $(0.9, 0.999)$；但 LLaMA [H. Touvron et al., 2023]、GPT-3 [T. B. Brown et al., 2020] 等大型语言模型常使用 $(0.9, 0.95)$。

算法如下，其中 ε 是一个很小的数（例如 10^{-8}），用于在 v 极小时改善 numerical stability。

> **Algorithm 1：AdamW Optimizer**
>
> 1. `init(θ)`：初始化 learnable parameters。
> 2. $m \leftarrow 0$：一阶矩向量初值，与 θ shape 相同。
> 3. $v \leftarrow 0$：二阶矩向量初值，与 θ shape 相同。
> 4. 对 $t = 1, \ldots, T$：
> 5. 从数据中采样 batch B_t。
> 6. $g \leftarrow \nabla_\theta \ell(\theta; B_t)$：计算 loss gradient。
> 7. $\alpha_t \leftarrow \alpha \dfrac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}$：计算第 t 次迭代的 adjusted α。
> 8. $\theta \leftarrow \theta - \alpha\lambda\theta$：应用 weight decay。

9. $m \leftarrow \beta_1 m + (1-\beta_1) g$：更新一阶矩估计。
10. $v \leftarrow \beta_2 v + (1-\beta_2) g^2$：更新二阶矩估计。
11. $\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v}+\varepsilon}$：应用经 moment 调整的权重更新。
12. 结束循环。

注意，t 从 1 开始。接下来实现该 optimizer。

题目（`adamw`）：实现 AdamW（2 分）

提交内容： 将 AdamW 实现为 `torch.optim.Optimizer` 的子类。类的 `__init__` 应接收 learning rate α，以及 β、ε、λ 等 hyperparameter。

为了保存 state，基类 `Optimizer` 提供 dictionary `self.state`，它把 `nn.Parameter` 对象映射到另一个 dictionary，可用于存储该 parameter 所需的任意信息。对于 AdamW，这些信息就是 moment estimate。

实现 `[adapters.get_adamw_cls]`，并确保通过：

```
uv run pytest -k test_adamw
```

题目（`adamw_accounting`）：使用 AdamW 训练的资源核算（2 分）

计算运行 AdamW 所需的内存和计算量。假设所有 tensor 均使用 `float32`。

(a) 运行 AdamW 所需的 peak memory 是多少？请按 parameters、activations、gradients 和 optimizer state 分解内存用量。答案应使用 `batch_size` 和模型 hyperparameter（`vocab_size`、`context_length`、`num_layers`、`d_model`、`num_heads`）表示。假设：

$$d_{ff} = \frac{8}{3} d_{model}.$$

为简化 activation memory 计算，只考虑以下组件：

- Transformer block：
  - RMSNorm；
  - multi-head self-attention sublayer：QKV projection、$QK^\top$ matrix multiplication、softmax、value 的 weighted sum、output projection；
  - position-wise feed-forward（SwiGLU）：W_1、W_2、gate branch 上的 SiLU、element-wise product、W_3。
- final RMSNorm；
- output embedding；
- logits 上的 cross-entropy。

提交内容： 分别给出 parameters、activations、gradients、optimizer state 的代数表达式，以及总和。

(b) 将答案代入 GPT-2 XL 规模模型，使表达式只依赖 `batch_size`。在 80 GB 内存中，最大 batch size 是多少？
提交内容： 形如 $a \cdot \text{batch_size} + b$ 的数值表达式，并给出最大 batch size。

(c) 执行一步 AdamW 需要多少 FLOPs？
提交内容： 一个代数表达式，并简要说明理由。

(d) Model FLOPs utilization（MFU）定义为：实测吞吐量（tokens per second）相对于硬件理论峰值 FLOP throughput 的比例 [A. Chowdhery et al., 2022]。

NVIDIA H100 GPU 对 “float32”（实际为 TensorFloat-32，原文称其实际上是 “bfloat19”）运算的理论峰值为 495 teraFLOP/s。假设能达到 50% MFU，在单张 H100 上以 batch size 1024 训练 GPT-2 XL 400K steps，需要多长时间？遵循 J. Kaplan 等人 [25] 和 J. Hoffmann 等人 [26]，假设 backward pass FLOPs 是 forward pass 的两倍。
提交内容： 训练所需小时数，并简要说明计算过程。

4.4 Learning Rate Scheduling

能让 loss 最快下降的 learning rate 往往会随训练阶段变化。训练 Transformer 时，通常使用 learning rate schedule：开始时使用较大的 learning rate，以快速更新；随着训练进行，再逐渐衰减到较小值。[8]

本作业实现训练 LLaMA [H. Touvron et al., 2023] 时使用的 cosine annealing schedule。

Scheduler 本质上只是一个函数：输入当前 step t 和其他相关参数（例如初始/最终 learning rate），返回 step t 的 gradient update 应使用的 learning rate。最简单的是 constant schedule，它对任何 t 都返回相同 learning rate。

Cosine annealing learning rate schedule 接收：

1. 当前 iteration t；
2. maximum learning rate α_{max}；
3. minimum/final learning rate α_{min}；
4. warm-up iterations 数量 T_w；
5. cosine annealing 的最终 iteration T_c。

第 t 次迭代的 learning rate 定义为：

Warm-up： 若 $t < T_w$，则：

$$\alpha_t = \frac{t}{T_w}\alpha_{max}.$$

Cosine annealing： 若 $T_w \le t \le T_c$，则：

$$\alpha_t = \alpha_{min} + \frac{1}{2}\left(1 + \cos\left(\frac{t - T_w}{T_c - T_w}\pi\right)\right)(\alpha_{max} - \alpha_{min}).$$

Post-annealing： 若 $t > T_c$，则：

$$\alpha_t = \alpha_{min}.$$

题目（`learning_rate_schedule`）：实现带 warmup 的 cosine learning rate schedule（1 分）

编写函数，接收 t、α_{max}、α_{min}、T_w 和 T_c，按上述 scheduler 返回 learning rate α_t。

然后实现 `[adapters.get_lr_cosine_schedule]`，并确保通过：

```
uv run pytest -k test_get_lr_cosine_schedule
```

4.5 Gradient Clipping

训练期间，有时会遇到产生巨大 gradient 的样本，导致训练不稳定。实践中常使用 gradient clipping 缓解该问题：在每次 backward pass 后、optimizer step 前，对 gradient norm 设置上限。

给定全部 parameter 的 gradient g，计算其 ℓ_2 norm $\|g\|_2$。若该 norm 小于 maximum value M，则保持 g 不变；否则，把 g 缩小：

$$g \leftarrow g \cdot \frac{M}{\|g\|_2 + \varepsilon},$$

其中加入很小的 ε（例如 10^{-6}）以提高 numerical stability。缩放后的 norm 会略小于 M。

题目（`gradient_clipping`）：实现 gradient clipping（1 分）

编写实现 gradient clipping 的函数。函数接收 parameter 列表和最大 ℓ_2 norm，并原地修改每个 parameter 的 gradient。使用 $\varepsilon = 10^{-6}$（PyTorch 默认值）。

实现适配器 `[adapters.run_gradient_clipping]`，并确保通过：

```
uv run pytest -k test_gradient_clipping
```

5 Training Loop

现在终于可以组合此前构建的主要组件：tokenized data、model 和 optimizer。

5.1 Data Loader

分词后的数据（例如在 `tokenizer_experiments` 中准备的数据）是一个 token 序列：

$$x = (x_1, \ldots, x_n).$$

即使源数据由多个独立文档组成（例如不同网页或 source code file），常见做法仍是把它们连接成单一 token 序列，并在文档之间加入 delimiter，例如 `<|endoftext|>` token。

Data loader 会把该序列转换为 batch stream。每个 batch 包含 B 个长度为 m 的序列，并与相应的长度同为 m 的 next-token target 配对。

例如，当 $B = 1$、$m = 3$ 时，下面是一种可能的 batch：

$$([x_2, x_3, x_4],\ [x_3, x_4, x_5]).$$

这种数据加载方式简化了训练：

1. 任意 $1 \le i \le n - m$ 都能产生有效 training sequence，因此采样极其简单。
2. 所有 training sequence 长度相同，无需 padding，可提高硬件利用率，也便于增大 batch size B。
3. 采样时无需加载完整数据集，因此容易处理无法整体装入内存的大型数据集。

题目（`data_loading`）：实现 data loading（2 分）

提交内容： 编写函数，接收：

- NumPy array x（存放 token ID 的整数 array）；
- `batch_size`；
- `context_length`；
- PyTorch device string，例如 `'cpu'` 或 `'cuda:0'`。

函数返回一对 tensor：采样得到的输入序列和对应的 next-token target。两者 shape 均应为 `(batch_size, context_length)`，内容为 token ID，并放置在请求的 device 上。

实现测试适配器 `[adapters.run_get_batch]`，然后运行：

```
uv run pytest -k test_get_batch
```

> **低资源提示：在 CPU 或 Apple Silicon 上加载数据**
> 如果计划在 CPU 或 Apple Silicon 上训练 LM，需要把数据移动到正确 device；之后模型也应使用同一 device。
> CPU 可使用 device string `'cpu'`；Apple Silicon（M 系列芯片）可使用 `'mps'`。
> MPS 相关资源：
>
> - `https://docs.pytorch.org/docs/stable/mps.html`
> - `https://docs.pytorch.org/docs/stable/notes/mps.html`
> - `https://developer.apple.com/documentation/metalperformanceshaders`

如果数据集太大，无法装入内存，应怎么办？可以使用名为 `mmap` 的 Unix system call。它会把磁盘文件映射到 virtual memory，并在访问相应内存位置时惰性加载文件内容，因此可以“假装”整个数据集都在内存中。

NumPy 通过 `np.memmap` 实现这一机制；如果 array 原先使用 `np.save` 保存，也可以在 `np.load` 时传入 `mmap_mode='r'`。返回的对象行为类似 NumPy array，在访问 entry 时按需加载。

训练期间从数据集（即 NumPy array）采样时，务必使用 memory-mapped mode 加载数据：根据保存方式，使用 `np.memmap`，或 `np.load(..., mmap_mode='r')`。还应指定与原 array 一致的 `dtype`。

建议显式检查 memory-mapped data 是否正确，例如确认其中没有超过预期 vocabulary size 的值。

5.2 Checkpointing

除加载数据外，训练过程中还需要保存模型。运行 job 时，我们常希望能够恢复中途停止的训练（例如 job timeout、机器故障等）。即使一切顺利，也可能希望后续访问 intermediate model，例如事后研究 training dynamics，或从训练不同阶段的模型中采样。

Checkpoint 应包含恢复训练所需的全部 state。至少要恢复 model weight。若使用 AdamW 等 stateful optimizer，还必须保存 optimizer state，例如 AdamW 的 moment estimate。为了恢复 learning rate schedule，还需要知道停止时的 iteration number。

PyTorch 很容易保存这些信息：

- 每个 `nn.Module` 都有 `state_dict()`，返回包含全部 learnable weight 的 dictionary；之后可用 `load_state_dict()` 恢复；
- 每个 `torch.optim.Optimizer` 同样支持 `state_dict()` / `load_state_dict()`；

- `torch.save(obj, dest)` 可把对象写入文件路径或 file-like object。对象可以是包含 tensor 的 dictionary，也可以包含整数等普通 Python object；之后用 `torch.load(src)` 重新加载。

题目（`checkpointing`）：实现模型 checkpoint（1 分）

实现以下两个函数，用于保存和加载 checkpoint。

```
def save_checkpoint(model, optimizer, iteration, out)
```

该函数应把 model、optimizer 和 iteration 的全部 state 写入 file-like object `out`。

可以使用 model 和 optimizer 的 `state_dict()` 获取相关 state，并使用 `torch.save(obj, out)` 把 `obj` 写入 `out`。PyTorch 在这里既支持路径，也支持 file-like object。

常见做法是让 `obj` 为 dictionary，但只要之后能够重新加载 checkpoint，可以使用任何格式。

参数：

```
model: torch.nn.Module
optimizer: torch.optim.Optimizer
iteration: int
out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]
```

```
def load_checkpoint(src, model, optimizer)
```

该函数从 `src`（路径或 file-like object）加载 checkpoint，并恢复 model 和 optimizer state。函数应返回 checkpoint 中保存的 iteration number。

可以使用 `torch.load(src)` 恢复 `save_checkpoint` 所保存的内容，再分别使用 model 和 optimizer 的 `load_state_dict()` 把它们恢复到之前的状态。

参数：

```
src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]
model: torch.nn.Module
optimizer: torch.optim.Optimizer
```

实现适配器 `[adapters.run_save_checkpoint]` 和 `[adapters.run_load_checkpoint]`，确保通过：

```
uv run pytest -k test_checkpointing
```

5.3 Training Loop

现在终于可以把已实现的全部组件组合进主训练脚本。

后续需要多次启动采用不同 hyperparameter 的训练 run，以研究不同选择如何影响训练。因此，应让启动不同配置尽可能方便，例如把 hyperparameter 作为 command-line argument。

题目（`training_together`）：组合所有组件（4 分）

提交内容： 编写一个 training script，在用户提供的输入上运行 training loop 并训练模型。建议至少支持：

- 配置和控制各种 model/optimizer hyperparameter；
- 使用 `np.memmap` 内存高效地加载大型训练集和 validation dataset；
- 把 checkpoint 序列化到用户提供的路径；
- 周期性记录 training/validation performance，例如输出到 console，和/或发送到 Weights & Biases 等外部服务。[9]

6 生成文本

既然已经可以训练模型，最后还需要让模型生成文本。

回顾一下，语言模型接收长度为 `sequence_length` 的整数序列（可能带 batch），产生大小为 `(sequence_length, vocab_size)` 的矩阵。序列中每个位置都对应一个概率分布，用于预测该位置之后的下一个 token。

下面编写若干函数，把该输出变成新序列的 sampling scheme。

Softmax

按照标准约定，语言模型直接输出 final linear layer 的结果，即 logits。因此，需要通过 Equation 10 中的 softmax，把 logits 转换成归一化概率。

Decoding

要从模型生成文本（decode），先给模型提供一段 prefix token sequence（即 prompt），让模型产生词表上的概率分布，预测序列中的下一个 token。然后从该词表分布中采样，确定下一个输出 token。

具体而言，decoding 的一步接收序列 $x_{1\dots t}$，并根据下式返回 token x_{t+1}：

$$P(x_{t+1} = i \mid x_{1\dots t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)}, \tag{21}$$

其中：

$$v = \mathrm{TransformerLM}(x_{1\dots t})_t \in \mathbb{R}^{vocab_size}. \tag{22}$$

`TransformerLM` 接收长度为 `sequence_length` 的序列，输出大小为 `(sequence_length, vocab_size)` 的矩阵。由于需要预测位置 t 之后的 token，因此取矩阵的最后一个元素。

反复从这些一步条件分布中采样，并把刚生成的 token 追加到下一 decoding timestep 的输入中，就得到基本 decoder。该过程持续到生成 end-of-sequence token `<|endoftext|>`，或达到用户指定的最大生成 token 数。

Decoder Tricks

本作业使用的是小模型，而小模型有时会生成质量很差的文本。两个简单 decoder trick 可改善这一问题。

第一种是 **temperature scaling**。使用 temperature parameter τ 修改 softmax：

$$\mathrm{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{vocab_size} \exp(v_j/\tau)}. \tag{23}$$

当 $\tau \to 0$ 时，v 中最大元素会占据主导，softmax 输出趋近于集中在该最大元素上的 one-hot vector。

第二种是 **nucleus sampling**，又称 **top-p sampling**，它通过截断低概率 token 修改 sampling distribution。

设 q 是经过 temperature-scaled softmax 得到的、大小为 `vocab_size` 的概率分布。带 hyperparameter p 的 nucleus sampling 按下式生成下一个 token：

$$P(x_{t+1} = i \mid q) = \begin{cases} \dfrac{q_i}{\sum_{j \in V(p)} q_j}, & i \in V(p), \\ 0, & \text{otherwise}, \end{cases} \tag{24}$$

其中 $V(p)$ 是满足：

$$\sum_{j \in V(p)} q_j \geq p$$

的最小索引集合。

可先按概率大小对 q 排序，然后从最大的 vocabulary element 开始选择，直到累计概率达到目标 p，从而容易地计算 $V(p)$。

题目（`decoding`）：Decoding（3 分）

提交内容： 实现从 language model 解码的函数。建议支持：

- 为用户提供的 prompt 生成 completion，即输入某个 $x_{1\dots t}$，持续采样 completion，直到遇到 `<|endoftext|>`；
- 允许用户控制最大生成 token 数；
- 给定 temperature，对预测的 next-token distribution 应用 softmax temperature scaling，再进行采样；
- 给定用户指定 threshold，执行 top-p sampling（[A. Holtzman et al., 2020]，也称 nucleus sampling）。

7 实验

现在把全部组件组合起来，在 pretraining dataset 上训练小型语言模型。

7.1 如何运行实验与提交结果

理解 Transformer 各架构组件设计理由的最佳方式，是亲自修改并运行模型。Hands-on experience 无可替代。

因此，必须能够快速、一致地实验，并记录所做工作：

- 为快速实验，将在小规模模型（总参数约 17M）和简单数据集 TinyStories 上运行大量实验；
- 为保持一致性，将系统性地 ablate 各组件并改变 hyperparameter；
- 为保留记录，需要提交 experiment log，以及每个实验对应的 learning curve。

为了能提交 loss curve，应周期性评估 validation loss，并同时记录 step 数和 wall-clock time。Weights & Biases 等 logging infrastructure 可能会有所帮助。

题目（`experiment_log`）：实验日志（3 分）

为 training/evaluation code 创建 experiment tracking infrastructure，使你可以基于 gradient step 和 wall-clock time 跟踪实验与 loss curve。

提交内容： 实验 logging infrastructure 代码，以及一个 experiment log 文档，记录本节后续题目中尝试过的所有内容。

7.2 TinyStories

首先使用非常简单的 TinyStories 数据集 [R. Eldan et al., 2023]。模型可在其上快速训练，同时也能观察到一些有趣行为。数据集获取说明见第 1 节。下面是一个数据样例。

> **示例（`tinystories_example`）：TinyStories 样例**
>
> 从前，有一个名叫 Ben 的小男孩。Ben 喜欢探索周围的世界。他看到了许多神奇的东西，例如商店里陈列的漂亮花瓶。一天，Ben 在商店里走着，发现了一个非常特别的花瓶。Ben 看到它时惊叹不已！他说："哇，这真是一个很棒的花瓶！我可以买下它吗？"店主微笑着说："当然可以。你可以把它带回家，让所有朋友看看它有多么漂亮！"于是 Ben 把花瓶带回家，为它感到非常自豪。他叫来朋友们，向他们展示这个漂亮的花瓶。所有朋友都觉得花瓶很美，简直不敢相信 Ben 有这么幸运。Ben 就这样在商店里找到了一个很棒的花瓶！

7.2.1 Hyperparameter Tuning

课程组提供一些基础 hyperparameter 作为起点，并要求你为其他参数寻找表现良好的设置。

- **Vocab size：10,000。** 典型词表大小为数万到数十万。可以改变该值，观察词表和模型行为如何变化。
- **Context length：256。** TinyStories 等简单数据集可能不需要长序列；但后续 OpenWebText 可能需要改变这一参数。尝试不同值，观察其对单 iteration runtime 和最终 perplexity 的影响。
- **`d_model`：512。** 比许多小型 Transformer 论文使用的 768 略小，但训练会更快。
- **`d_ff`：1344。** 约为 $\frac{8}{3}d_{model}$，同时是 64 的倍数，有利于 GPU performance。
- **RoPE theta parameter Θ：10,000。**
- **层数和 heads：4 layers、16 heads。** 合计约 17M 个 non-embedding parameter，是相当小的 Transformer。
- **总处理 token 数：327,680,000。** `batch size × total step count × context length` 应大致等于该值。

还需要通过 trial and error，为以下 hyperparameter 寻找良好默认值：learning rate、learning rate warmup、其他 AdamW hyperparameter（β_1、β_2、ε）以及 weight decay。D. P. Kingma 等人 [22] 中可以找到这些参数的典型选择。

7.2.2 组合训练流程

现在可以把训练好的 BPE tokenizer、已分词的 training dataset 和所编写的 training loop 组合起来。

重要说明：若实现正确且高效，上述 hyperparameter 在 1 张 B200 GPU 上的 runtime 应约为 20-30 分钟。若耗时明显更长，请检查 data loading、checkpointing 或 validation loss 代码是否成为瓶颈，并确认实现已正确 batching。

7.2.3 调试模型架构的技巧

强烈建议熟悉 IDE 内置 debugger（例如 VSCode/Zed）。与通过 `print` 语句调试相比，它能节省大量时间。若使用文本编辑器，可以使用 `ipdb` 等工具。

调试 model architecture 时，还有一些良好实践：

- 开发任何 neural net architecture 时，常见第一步是让模型 overfit 单个 minibatch。若实现正确，应能很快把 training loss 降到接近 0。
- 在不同 model component 中设置 debug breakpoint，检查 intermediate tensor shape 是否符合预期。
- 监控 activation、model weight 和 gradient 的 norm，确认它们没有 exploding 或 vanishing。

题目（ `learning_rate` ）：调整 learning rate（2 B200 小时，3 分）

Learning rate 是最重要的 hyperparameter 之一。基于已经训练的 base model，回答：

(a) 对 learning rate 执行 hyperparameter sweep，报告最终 loss；若 optimizer diverge，则注明。
提交内容： 多个 learning rate 对应的 learning curve，并解释 hyperparameter search strategy。

额外提交目标： 一个 TinyStories validation loss（per-token）不高于 1.45 的模型。

> **低资源提示：在 CPU 或 Apple Silicon 上训练少量 step**
> 如果使用 `cpu` 或 `mps`，应把总处理 token 数降到 40,000,000；这已足以生成较流畅文本。也可以把目标 validation loss 从 1.45 放宽到 2.00。
> 课程组参考实现在 M4 Max、36 GB RAM 上使用：
>
> ```
> batch size × total step count × context length
> = 32 × 5000 × 256
> = 40,960,000 tokens
> ```
>
> CPU 耗时 1 小时 22 分钟，MPS 耗时 36 分钟。在 step 5000 时，validation loss 为 1.80。
> 其他提示：
>
> - 使用 N 个 training step 时，建议调整 cosine learning rate decay schedule，使其恰好在 step N 完成衰减，即达到 minimum learning rate。
> - 使用 MPS 时不要启用 TF32 kernel，即不要像 CUDA device 上有时会做的那样设置：
>
>   ```
>   torch.set_float32_matmul_precision('high')
>   ```
>
>   课程组在 MPS（torch 2.9.0）上尝试启用 TF32 kernel 时发现，backend 有时会静默使用损坏的 kernel，导致训练不稳定。

> - 可使用 `torch.compile` 对模型进行 JIT compilation，以加快训练：
>   - 在 CPU 上：
>
>     ```
>     model = torch.compile(model)
>     ```
>
>   - 在 MPS 上，可使用以下方式对 backward pass 做一定优化：
>
>     ```
>     model = torch.compile(model, backend="aot_eager")
>     ```
>
>   截至 torch 2.9.0，MPS 尚不支持使用 Inductor 进行 compilation。

(b) 一种经验观点认为，最佳 learning rate 位于“稳定性边缘（edge of stability）”。研究 learning rate 开始 diverge 的临界点与最佳 learning rate 之间的关系。
提交内容： 一组逐渐增大 learning rate 的 learning curve，其中至少包含一次 divergent run；并分析这与 convergence rate 的关系。

接下来改变 batch size，观察训练会发生什么。Batch size 很重要：通过执行更大的 matrix multiplication，它能提高 GPU efficiency。但 batch size 是否总是越大越好？通过实验进行验证。

题目（ `batch_size_experiment` ）：Batch size 变化（1 B200 小时，1 分）

把 batch size 从 1 一直改变到 GPU memory limit。至少尝试若干中间值，包括 64、128 等常见大小。

提交内容： 不同 batch size run 的 learning curve；如有必要，应重新优化 learning rate。
提交内容： 用几句话讨论 batch size 及其对训练影响的发现。

有了 decoder 后，现在可以生成文本。使用模型生成，并观察质量。作为参考，输出应至少达到下面示例的水平。

> **示例（ `ts_generate_example` ）：TinyStories 语言模型的样例输出**
>
> 从前，有一个名叫 Lily 的漂亮女孩。她喜欢吃口香糖，尤其是那块大大的黑色口香糖。一天，Lily 的妈妈请她帮忙做晚饭。Lily 非常兴奋！她很喜欢帮助妈妈。Lily 的妈妈为晚饭煮了一大锅汤。Lily 很开心，说：“谢谢你，妈妈！我爱你。”她帮妈妈把汤倒进一个大碗里。晚饭后，Lily 的妈妈又做了一些美味的汤。Lily 非常喜欢！她说：“谢谢你，妈妈！这汤太好喝了！”妈妈微笑着说：“我很高兴你喜欢，Lily。”她们做完饭，又继续一起做饭。故事结束。

> **低资源提示：在 CPU 或 Apple Silicon 上生成文本**
> 如果使用只处理 40M token 的低资源配置，生成结果仍应看起来像英语，但流畅度可能不如上例。课程组在 40M token 上训练的 TinyStories LM 样例输出如下（保留英文原文，以呈现模型真实错误）：
>
> ```
> Once upon a time, there was a little girl named Sue. Sue had a tooth that she loved very much. It
> was his best head. One day, Sue went for a walk and met a ladybug! They became good friends
> and played on the path together.
> “Hey, Polly! Let’s go out!” said Tim. Sue looked at the sky and saw that it was difficult to find a
> way to dance shining. She smiled and agreed to help the talking!“
> As Sue watched the sky moved, what it was. She
> ```

下面是精确题目要求。

题目（`generate`）：生成文本（1 分）

使用 decoder 和训练好的 checkpoint，报告模型生成的文本。可能需要调整 temperature、top-p 等 decoder parameter，才能得到流畅输出。

提交内容： 至少 256 token 的文本输出（或生成到第一个 `<|endoftext|>` 为止），并简要评论输出流畅度，以及至少两个影响输出质量好坏的因素。

7.3 Ablation 与架构修改

理解 Transformer 的最佳方式，是实际修改它并观察行为。下面进行若干简单 ablation 和 modification。

Ablation 1：Layer normalization

人们常说 layer normalization 对 Transformer 训练稳定性很重要。但也许我们想冒一点险。移除每个 Transformer block 中的 RMSNorm，看看会发生什么。

题目（`layer_norm_ablation`）：移除 RMSNorm 并训练（0.5 B200 小时，1 分）

移除 Transformer 中全部 RMSNorm 后进行训练。在此前最佳 learning rate 下会发生什么？通过使用更低 learning rate，能否恢复稳定性？

提交内容： 移除 RMSNorm 后训练的 learning curve，以及最佳 learning rate 对应的 learning curve。
提交内容： 用几句话评论 RMSNorm 的影响。

接下来研究另一个乍看似乎随意的 layer normalization 选择。Pre-norm Transformer block 定义为：

$$z = x + \mathrm{MultiHeadSelfAttention}(\mathrm{RMSNorm}(x)), \tag{25}$$

$$y = z + \mathrm{FFN}(\mathrm{RMSNorm}(z)). \tag{26}$$

这是对原始 Transformer 架构少数已形成“共识”的修改之一。原始架构采用 post-norm：

$$z = \mathrm{RMSNorm}\left(x + \mathrm{MultiHeadSelfAttention}(x)\right), \tag{27}$$

$$y = \mathrm{RMSNorm}\left(z + \mathrm{FFN}(z)\right). \tag{28}$$

现在恢复 post-norm，观察结果。

题目（`pre_norm_ablation`）：实现 post-norm 并训练（0.5 B200 小时，1 分）

把 pre-norm Transformer 实现改为 post-norm，使用 post-norm 模型训练并观察结果。

提交内容： post-norm Transformer 的 learning curve，并与 pre-norm 对比。

由此可以看到，layer normalization 对 Transformer 行为有重大影响，甚至其放置位置也非常重要。

Ablation 2：Position embeddings

接下来研究 position embedding 对模型性能的影响。具体而言，把使用 RoPE 的 base model 与完全不使用 position embedding 的模型（NoPE）进行比较。

事实证明，decoder-only Transformer——即使用本作业所实现 causal mask 的模型——理论上即使不显式提供 position embedding，也可以推断 relative 或 absolute position information [Y.-H. H. Tsai et al., 2019; A. Kazemnejad et al., 2023]。现在通过实验比较 NoPE 与 RoPE。

题目（`no_pos_emb`）：实现 NoPE（0.5 B200 小时，1 分）

修改使用 RoPE 的 Transformer 实现，彻底移除 position embedding information，观察结果。

提交内容： 比较 RoPE 与 NoPE 性能的 learning curve。

Ablation 3：SwiGLU vs. SiLU

接下来遵循 N. Shazeer [20]，检验 feed-forward network 中 gating 的重要性：比较 SwiGLU feed-forward network 与使用 SiLU activation、但不使用 GLU 的 feed-forward network：

$$\mathrm{FFN}_{\mathrm{SiLU}}(x) = W_2\,\mathrm{SiLU}(W_1 x). \tag{29}$$

回顾一下，在 SwiGLU 实现中，inner feed-forward layer 的维度约为：

$$d_{ff} = \frac{8}{3} d_{model},$$

同时确保 $d_{ff} \bmod 64 = 0$，以利用 GPU tensor core。

在这个 ablation baseline 中，`FFNSiLU` 应改用：

$$d_{ff} = 4 d_{model},$$

从而使参数量与默认 SwiGLU feed-forward network 大致匹配；后者有三个 weight matrix，而不是两个。

题目（`swiglu_ablation`）：SwiGLU vs. SiLU（0.5 B200 小时，1 分）

提交内容： 在参数量大致匹配的情况下，比较 SwiGLU 和 SiLU feed-forward network 性能的 learning curve。
提交内容： 用几句话讨论实验发现。

> **低资源提示：GPU 资源有限的在线学习者应在 TinyStories 上测试修改**
> 作业剩余部分将转向规模更大、噪声更多的 web dataset OpenWebText，进行架构修改实验，并可选提交课程排行榜。
> 在 OpenWebText 上把 LM 训练到流畅需要很长时间，因此建议 GPU 资源有限的在线学习者继续在 TinyStories 上测试修改，并使用 validation loss 作为性能指标。

7.4 在 OpenWebText 上运行

现在转向一个由 web crawl 构建、更标准的 pretraining dataset。课程提供了 OpenWebText [A. Gokaslan et al., 2019] 的一个小型样本，存放在单个文本文件中；访问方法见第 1 节。

下面是一个 OpenWebText 示例。可以看到，其文本比 TinyStories 真实、复杂且多样得多。建议浏览 training dataset，了解 web-scraped corpus 的训练数据是什么样子。

> **示例（`owt_example`）：OWT 样例**
>
> Baseball Prospectus 技术总监 Harry Pavlidis 雇用 Jonathan Judge 时冒了一个险。
>
> Pavlidis 知道，正如 Alan Schwarz 在 *The Numbers Game* 中所写：“美国文化中，没有哪个角落比棒球运动员的表现得到更精确的计数、更热情的量化。”只需在这里那里点击几下，你就能查到 Noah Syndergaard 的快速球在飞向本垒板的过程中每分钟旋转超过 2,100 次，Nelson Cruz 在 2016 年符合资格的击球手中拥有全联盟最高的平均击球初速，以及无数其他仿佛来自电子游戏或科幻小说的细节。不断上涨的数据海洋赋予了棒球文化中一个日益重要的角色更多力量：业余数据分析者。
>
> 这种力量也带来了更多审视——既针对测量本身，也针对测量背后的人和出版机构。在 Baseball Prospectus 工作时，Pavlidis 非常清楚量化结果不完美会引发怎样的反弹。他也知道网站的接球指标需要重新设计，而完成这项工作需要一个学识深厚的人——能够处理复杂统计建模问题的人。
>
> “他让我们感到害怕。”——Harry Pavlidis
>
> 根据 Judge 的文章，以及两人在一次由网站赞助的球场活动中的互动，Pavlidis 直觉认为 Judge “懂这一套”。[…]

注意：本实验可能需要重新调整 learning rate、batch size 等 hyperparameter。

题目（`main_experiment`）：在 OWT 上实验（2 B200 小时，2 分）

使用与 TinyStories 相同的 model architecture 和总 training iteration，在 OpenWebText 上训练 language model。模型表现如何？

提交内容： OpenWebText LM 的 learning curve。描述其 loss 与 TinyStories 的差异——应如何理解这些 loss?
提交内容： OpenWebText LM 的生成文本，格式与 TinyStories 输出相同。文本流畅度如何？为什么在模型和 compute budget 都与 TinyStories 相同的情况下，输出质量反而更差?

7.5 自定义修改与排行榜

恭喜完成到这里，作业接近结束。现在尝试改进 Transformer architecture，并观察自己的 hyperparameter 和 architecture 与班上其他同学相比表现如何。

排行榜规则

除以下限制外，没有其他限制：

- **Runtime：** 提交程序在 B200 上最多运行 45 分钟。如果使用 SLURM 或 Modal，可以考虑在 submission script 中强制这一限制。
- **Data：** 只能使用课程提供的 OpenWebText training dataset。

除此之外，可以自由尝试任何方法。

若需要实现思路，可参考：

- 当前先进的 open-source LLM family，例如 Llama 3 [A. Grattafiori et al., 2024] 或 Qwen 2.5 [A. Yang et al., 2024]。
- NanoGPT speedrun 仓库： `github.com/KellerJordan/modded-nanogpt` 。社区成员在其中分享许多用于“速通”小规模 language model pretraining 的有趣修改。

例如，一个可追溯到原始 Transformer 论文的常见修改，是把 input embedding 和 output embedding 的权重绑定（weight tying）；参见 A. Vaswani 等人 [8] 第 3.4 节和 A. Chowdhery 等人 [16] 第 2 节。若尝试 weight tying，可能需要减小 embedding/LM head 初始化的 standard deviation。

在进行完整的 45 分钟 run 之前，应先在 OpenWebText 小子集或 TinyStories 上测试这些修改。

需要提醒的是，排行榜上效果良好的某些修改，可能无法泛化到更大规模 pretraining。课程的 scaling laws 单元会进一步探讨这一点。

题目（ `leaderboard` ）：排行榜（10 B200 小时，6 分）

按照上述排行榜规则训练模型，目标是在 **0.75 B200-hours** 的预算内尽可能降低 language model 的 validation loss。

提交内容：

- 最终记录的 validation loss；
- 对应的 learning curve，必须清楚地使用 wall-clock time 作为 x-axis，并显示总时间少于 45 分钟；
- 对所采用方法的说明。

排行榜提交至少应击败 validation loss 为 5.0 的朴素 baseline。提交地址：

```
github.com/stanford-cs336/assignment1-basics-leaderboard
```

参考文献

> 为便于检索，论文标题、会议名称和链接保留英文原文。

[1] R. Eldan and Y. Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?.” 2023.

[2] A. Gokaslan, V. Cohen, E. Pavlick, and S. Tellex, “OpenWebText corpus.” 2019.

[3] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in *Proc. of ACL*, 2016.

[4] C. Wang, K. Cho, and J. Gu, “Neural Machine Translation with Byte-Level Subwords.” 2019.

[5] P. Gage, “A new algorithm for data compression,” *C Users Journal*, vol. 12, no. 2, pp. 23–38, Feb. 1994.

[6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners.” 2019.

[7] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training.” 2018.

[8] A. Vaswani et al., “Attention is All you Need,” in *Proc. of NeurIPS*, 2017.

[9] T. Q. Nguyen and J. Salazar, “Transformers without Tears: Improving the Normalization of Self-Attention,” in *Proc. of IWSWLT*, 2019.

[10] R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” in *Proc. of ICML*, 2020.

[11] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization.” 2016.

[12] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models.” 2023.

[13] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” in *Proc. of NeurIPS*, 2019.

[14] A. Grattafiori et al., “The Llama 3 Herd of Models.” [Online]. Available: `https://arxiv.org/abs/2407.21783`

[15] A. Yang et al., “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.

[16] A. Chowdhery et al., “PaLM: Scaling Language Modeling with Pathways.” 2022.

[17] D. Hendrycks and K. Gimpel, “Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units.” 2016.

[18] S. Elfwing, E. Uchibe, and K. Doya, “Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning.” [Online]. Available: `https://arxiv.org/abs/1702.03118`

[19] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language Modeling with Gated Convolutional Networks.” [Online]. Available: `https://arxiv.org/abs/1612.08083`

[20] N. Shazeer, “GLU Variants Improve Transformer.” 2020.

[21] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2021.

[22] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. of ICLR*, 2015.

[23] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. of ICLR*, 2019.

[24] T. B. Brown et al., “Language Models are Few-Shot Learners,” in *Proc. of NeurIPS*, 2020.

[25] J. Kaplan et al., “Scaling Laws for Neural Language Models.” 2020.

[26] J. Hoffmann et al., “Training Compute-Optimal Large Language Models.” 2022.

[27] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” in *Proc. of ICLR*, 2020.

[28] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, “Transformer Dissection: An Unified Understanding for Transformer
`s Attention via the Lens of Kernel,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th Interna`
10.18653/v1/D19-1443`.

[29] A. Kazemnejad, I. Padhi, K. Natesan, P. Das, and S. Reddy, “The Impact of Positional Encoding on Length Generalization in Transformers,” in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. [Online]. Available: `https://openreview.net/forum?id=Drrl2gcjzl`

1. 完整列表见 `pytorch.org/docs/stable/nn.html#containers`。 ↩
2. 原始 BPE 方案 [R. Sennrich et al., 2016] 要求加入 end-of-word token。本作业训练字节级 BPE 时不加入 end-of-word token，因为所有字节——包括空白和标点——都已包含在词表中。由于空格和标点被显式表示，学到的 BPE merges 会自然反映词边界。 ↩
3. 关于 Unicode replacement character 的更多信息，见 `en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character`。 ↩
4. `einops` 得到了广泛使用，而 `einx` 尚未经过同等程度的实战检验。如果发现 `einx` 有限制或 bug，可以改用 `einops` 加普通 PyTorch。 ↩
5. 作为 stretch goal，可把 key、query、value projection 合并成一个 weight matrix，从而只需一次 matrix multiplication。 ↩
6. $o_i[k]$ 表示向量 o_i 在索引 k 处的值。 ↩
7. 这对应于 target x_{i+1} 上的 Dirac delta distribution 与预测分布 $\mathrm{softmax}(o_i)$ 之间的 cross-entropy。 ↩
8. 有时也会使用让 learning rate 再次升高（restart）的 schedule，以帮助越过 local minimum。 ↩
9. `wandb.ai` ↩